

```
1 require('./minigame/pixi-adapter/index');
2 require('./minigame/game-bundle.js');
3 /**
4  * 无尽纹章 - 小游戏入口 (须先于 PIXI.Application 加载 unsafe-eval patch)
5  */
6 import '@core/pixiUnsafeEvalPatch';
7 import { createPixiHost } from '@boot/createPixiApp';
8 import { AssetLoader } from '@core/AssetLoader';
9 import { GAME_KEY, GAME_TITLE } from '@config/gameKey';
10 import { GameFlow } from '@view/GameFlow';
11 import '@platform/wxPlatform';
12 declare const GameGlobal: any;
13 function formatBootErr(e: unknown): string {
14   if (e == null) return String(e);
15   if (typeof e === 'string') return e;
16   if (e instanceof Error) return e.stack || e.message;
17   if (typeof e === 'object') {
18     const o = e as { errMsg?: unknown; message?: unknown; reason?: unknown };
19     if (o.errMsg != null) return String(o.errMsg);
20     if (o.message != null) return String(o.message);
21     if (o.reason != null) return formatBootErr(o.reason);
22   }
23   try {
24     return JSON.stringify(e);
25   } catch {
26     return Object.prototype.toString.call(e);
27   }
28   return String(e);
29 }
30 if (typeof GameGlobal !== 'undefined') {
31   GameGlobal.onError = (msg: unknown) => {
32     console.error('[GlobalError]', formatBootErr(msg));
33   };
34   GameGlobal.onUnhandledRejection = (ev: unknown) => {
35     console.error('[UnhandledRejection]', formatBootErr(ev));
36   };
37 }
38 function boot(): void {
39   const canvas =
40     (typeof GameGlobal !== 'undefined' && GameGlobal.canvas) ||
41     (typeof window !== 'undefined' && (window as unknown as { canvas?: HTMLCanvasElement }).canva...
42   null;
43   if (!canvas) {
44     console.error('[main] 无法获取 canvas, 请确认 pixi-adapter 已加载');
45     return;
46   }
47   const host = createPixiHost(canvas);
48   new GameFlow(host);
49   try {
50     host.renderer.render(host.stage);
```

```
51 } catch (e) {
52   console.error('[main] 首次 render 失败:', e);
53 }
54 console.log('[main] ${GAME_TITLE} (${GAME_KEY}) MVP 启动, screen:', host.screen.width, 'x', host....
55 void AssetLoader.prefetchManifest();
56 }
57 /** 微信首帧 canvas 尺寸偶发未就绪: 双 rAF 再启动 (huahua 类项目常用) */
58 requestAnimationFrame() => {
59   requestAnimationFrame() => {
60     try {
61       boot();
62     } catch (e) {
63       console.error('[main] boot 异常:', e);
64     }
65   };
66 };
67 /**
68  * 现在只导出 GameKey, 保证全项目统一引用。
69  */
70 import { GAME_KEY } from '@config/gameKey';
71 export { GAME_KEY };
72 /** 经分 ingest 端点 (与 xiao_chu 共用 CloudBase 环境) */
73 export const ANALYTICS_ENDPOINT =
74   'https://rosa-env-d7grf78r5dbd37323.service.tcloudbase.com/analytics-ingest/track';
75 import type { UnitArchetypeDef, UnitDef, UnitKind, UnitState, Vec2 } from './types';
76 import { effectiveUnitDef } from './effectiveUnit';
77 import { cellsFromDist, reachableCells } from './path';
78 import { manhattan } from './grid';
79 import { computeDamage, counterMultiplier } from './damage';
80 import type { TerrainGrid } from './grid';
81 import { getTerrainAt } from './grid';
82 import { getTerrainSpec } from '@data/terrainSpec';
83 export type AiDifficulty = 'easy' | 'normal' | 'hard';
84 function key(p: Vec2): string {
85   return `${p.x},${p.y}`;
86 }
87 function living(units: UnitState[]): UnitState[] {
88   return units.filter((u) => u.hp > 0);
89 }
90 function defOf(u: UnitState, defs: Record<UnitKind, UnitArchetypeDef>): UnitDef {
91   return effectiveUnitDef(u, defs);
92 }
93 /**
94  * 普攻能否从 `from` 打到格子 `cell` (近战邻格 / 远程 1..range) 。
95  *
96  * 只要 `isRanged` 与 `range` 两个字段, 不要求完整 `UnitDef`: 威胁染色只有射程信息,
97  * 没有也不需要凑出一整个单位定义。
98  */
99 export function canAttackCell(
100   atkDef: Pick<UnitDef, 'isRanged' | 'range'>,
```

```

101   from: Vec2,
102   cell: Vec2,
103 ): boolean {
104   const d = manhattan(from, cell);
105   if (atkDef.isRanged) return d >= 1 && d <= atkDef.range;
106   return d === 1;
107 }
108 export function canAttackFrom(
109   atkDef: UnitDef,
110   fromPos: Vec2,
111   target: UnitState,
112 ): boolean {
113   return canAttackCell(atkDef, fromPos, target.pos);
114 }
115 /**
116  * 从 `fromPos` 出发要打谁; 射程内无人返回 null。
117  *
118  * 导出是给引擎的「续打」用的: 人工模式按下跳过时, 单位可能已经由玩家走到某格了,
119  * 这时不能再用 `chooseTurnAction` 的结果--那个目标是配着它想去的**另一格**算出来的,
120  * 从玩家选的位置未必打得到, 续打就会莫名少一刀。
121  */
122 export function selectAttackTarget(
123   atkDef: UnitDef,
124   fromPos: Vec2,
125   foes: UnitState[],
126   defs: Record<UnitKind, UnitArchetypeDef>,
127   difficulty: AiDifficulty,
128 ): UnitState | null {
129   const alive = living(foes);
130   const inRange = alive.filter((t) => canAttackFrom(atkDef, fromPos, t));
131   if (inRange.length === 0) return null;
132   const taunters = inRange.filter((t) => defOf(t, defs).taunt);
133   if (taunters.length > 0) {
134     return taunters.reduce((a, b) => (a.hp <= b.hp ? a : b));
135   }
136   if (difficulty === 'easy') {
137     return inRange[Math.floor(Math.random() * inRange.length)];
138   }
139   if (difficulty === 'hard') {
140     let bestScore = -Infinity;
141     let best: UnitState = inRange[0];
142     for (const t of inRange) {
143       const cm = counterMultiplier(atkDef.id, t.defId);
144       const killBonus = t.hp <= atkDef.atk * cm ? 100 : 0;
145       const score = cm * 10 + killBonus - t.hp * 0.1;
146       if (score > bestScore) {
147         bestScore = score;
148         best = t;
149       }
150     }

```

```
151   return best;
152 }
153 return inRange.reduce((a, b) => (a.hp <= b.hp ? a : b));
154 }
155 export interface TurnChoice {
156   moveTo: Vec2 | null;
157   attackTarget: UnitState | null;
158 }
159 function evaluateCell(
160   cell: Vec2,
161   self: UnitState,
162   atkDef: UnitDef,
163   allUnits: UnitState[],
164   defs: Record<UnitKind, UnitArchetypeDef>,
165   terrain: TerrainGrid,
166   difficulty: AiDifficulty,
167 ): { score: number; target: UnitState | null } {
168   const foes = living(allUnits).filter((u) => u.faction !== self.faction);
169   const t = selectAttackTarget(atkDef, cell, foes, defs, difficulty);
170   const dmg = t ? computeDamage(atkDef, defOf(t, defs), terrain, cell, t.pos) : 0;
171   let score = dmg;
172   if (difficulty === 'hard') {
173     const tSpec = getTerrainSpec(getTerrainAt(terrain, cell));
174     score += tSpec.atkMul > 1 ? 5 : 0;
175     score += tSpec.defMul < 1 ? 3 : 0;
176     score -= tSpec.dotPerRound > 0 ? 4 : 0;
177     if (t && t.hp <= dmg) score += 20;
178   }
179   return { score, target: t };
180 }
181 export function chooseTurnAction(
182   self: UnitState,
183   defs: Record<UnitKind, UnitArchetypeDef>,
184   allUnits: UnitState[],
185   terrain: TerrainGrid,
186   difficulty: AiDifficulty = 'normal',
187 ): TurnChoice {
188   if (difficulty === 'easy' && Math.random() < 0.15) {
189     return { moveTo: null, attackTarget: null };
190   }
191   const atkDef = defOf(self, defs);
192   const blocked = new Set(
193     living(allUnits)
194       .filter((u) => u.uid !== self.uid)
195       .map((u) => key(u.pos)),
196   );
197   const dist = reachableCells(self.pos, atkDef.move, blocked, terrain);
198   const candidates = cellsFromDist(self.pos, dist);
199   let bestScore = -1;
200   let bestMoveCost = 99;
```

```

201 let bestCell: Vec2 = self.pos;
202 let bestTarget: UnitState | null = null;
203 for (const cell of candidates) {
204   const moveCost = manhattan(cell, self.pos);
205   const { score, target } = evaluateCell(cell, self, atkDef, allUnits, defs, terrain, difficulty);
206   if (score > bestScore || (score === bestScore && score > 0 && moveCost < bestMoveCost)) {
207     bestScore = score;
208     bestMoveCost = moveCost;
209     bestCell = cell;
210     bestTarget = target;
211   }
212 }
213 if (bestScore > 0 && bestTarget) {
214   const moveTo = manhattan(bestCell, self.pos) === 0 ? null : bestCell;
215   return { moveTo, attackTarget: bestTarget };
216 }
217 const enemies = living(allUnits).filter((u) => u.faction !== self.faction);
218 if (enemies.length === 0) return { moveTo: null, attackTarget: null };
219 // 打不到任何人时朝最近的敌人走
220 const nearest = enemies.reduce((a, b) =>
221   (manhattan(self.pos, a.pos) <= manhattan(self.pos, b.pos) ? a : b));
222 let bestDist = Infinity;
223 let walk: Vec2 | null = null;
224 for (const cell of candidates) {
225   if (manhattan(cell, self.pos) === 0) continue;
226   const d = manhattan(cell, nearest.pos);
227   if (d < bestDist) {
228     bestDist = d;
229     walk = cell;
230   }
231 }
232 return { moveTo: walk, attackTarget: null };
233 }
234 /** 三角克制：攻击方 kind -> 被优先进攻的 kind（若目标为该 kind 则伤害 x） */
235 export const COUNTER_STRONG = 1.25;
236 export const COUNTER_WEAK = 0.85;
237 /** 回合上限：控制单场时长（超时判负），也防止死循环 */
238 export const MAX_BATTLE_ROUNDS = 30;
239 /** 玩家部署在战场最下两行（行号从 0 起，随关卡 `terrain` 高度变化） */
240 export function playerDeployRowRange(gridHeight: number): readonly [number, number] {
241   const h = Math.max(2, gridHeight);
242   return [h - 2, h - 1] as const;
243 }
244 import type { UnitDef, UnitKind, Vec2 } from './types';
245 import { COUNTER_STRONG, COUNTER_WEAK } from './constants';
246 import { getTerrainAt, type TerrainGrid } from './grid';
247 import { getTerrainSpec } from '@data/terrainSpec';
248 /** 骑 -> 剑 -> 弓 -> 骑；盾卫不参与 */
249 export function counterMultiplier(attacker: UnitKind, target: UnitKind): number {
250   if (attacker === 'shield' || target === 'shield') return 1;

```

```
251 const strong = (  
252   (attacker === 'cavalry' && target === 'sword')  
253   || (attacker === 'sword' && target === 'bow')  
254   || (attacker === 'bow' && target === 'cavalry')  
255 );  
256 const weak = (  
257   (attacker === 'sword' && target === 'cavalry')  
258   || (attacker === 'bow' && target === 'sword')  
259   || (attacker === 'cavalry' && target === 'bow')  
260 );  
261 if (strong) return COUNTER_STRONG;  
262 if (weak) return COUNTER_WEAK;  
263 return 1;  
264 }  
265 export function terrainAttackMul(  
266   terrainGrid: TerrainGrid,  
267   attackerPos: Vec2,  
268 ): number {  
269   const tid = getTerrainAt(terrainGrid, attackerPos);  
270   return getTerrainSpec(tid).atkMul;  
271 }  
272 export function terrainDefenseMul(  
273   terrainGrid: TerrainGrid,  
274   targetPos: Vec2,  
275 ): number {  
276   const tid = getTerrainAt(terrainGrid, targetPos);  
277   return getTerrainSpec(tid).defMul;  
278 }  
279 /**  
280  * 攻击方所站地形对本次伤害的影响，供回放飘字用；无影响返回 null。  
281  *  
282  * 文案从 spec 现算而不是写死：改了 `atkMul` 忘了改文案，玩家看到的数字和飘字就会对不上，  
283  * 而这类不一致比没有飘字更伤--它教会玩家不要相信 UI。  
284  */  
285 export function terrainAttackNote(terrainGrid: TerrainGrid, attackerPos: Vec2): string | null {  
286   const spec = getTerrainSpec(getTerrainAt(terrainGrid, attackerPos));  
287   if (spec.atkMul === 1) return null;  
288   return `${spec.name} ${formatPct(spec.atkMul)}%`;  
289 }  
290 /** 目标所站地形对本次伤害的影响，供回放飘字用；无影响返回 null */  
291 export function terrainDefenseNote(terrainGrid: TerrainGrid, targetPos: Vec2): string | null {  
292   const spec = getTerrainSpec(getTerrainAt(terrainGrid, targetPos));  
293   if (spec.defMul === 1) return null;  
294   return `${spec.name} ${formatPct(spec.defMul)}%`;  
295 }  
296 function formatPct(mul: number): string {  
297   const pct = Math.round((mul - 1) * 100);  
298   return pct > 0 ? `+${pct}%` : `-${pct}%`;  
299 }  
300 /**
```

```
301 * 基础伤害: `atk` × 克制 × 攻击方地形 × 目标地形`, 下限 1。
302 *
303 * `targetPos` 现在是必填。它曾经是可选的, 于是技能伤害那条路径一直没传,
304 * 技能等于无视目标地形--只要有一种可通行地形带减伤, 同一个森林里的敌人就会
305 * 「普攻打不动、技能照样打满」。地形规则必须对普攻和技能完全一致,
306 * 不然「站进森林」这条策略是真是假取决于对手用哪一招, 没人教得会。
307 */
308 export function computeDamage(
309   attackerDef: UnitDef,
310   targetDef: UnitDef,
311   terrainGrid: TerrainGrid,
312   attackerPos: Vec2,
313   targetPos: Vec2,
314 ): number {
315   const cm = counterMultiplier(attackerDef.id, targetDef.id);
316   const tm = terrainAttackMul(terrainGrid, attackerPos);
317   const dm = terrainDefenseMul(terrainGrid, targetPos);
318   const raw = attackerDef.atk * cm * tm * dm;
319   return Math.max(1, Math.floor(raw));
320 }
321 import type { UnitArchetypeDef, UnitDef, UnitKind, UnitState } from './types';
322 import { defaultSkillId, skillDefForId } from '@data/skillCatalog';
323 import {
324   sumTimedAtkBonus,
325   sumTimedAtkDown,
326   sumTimedSpdBonus,
327   sumTimedSpdDown,
328   timedTauntActive,
329 } from './timedBattleEffects';
330 /**
331 * 合并兵种 `base`/`strike`、上场覆盖与限时 buff/debuff。
332 *
333 * **技能来源按阵营分叉**：
334 * - 我方: `battleSkill`, 没有则回退 `defaultSkillId(defId)` (职业默认技)
335 * - 敌方: **只认显式 `battleSkill`**, 没有就是纯普攻
336 *
337 * 敌方曾经也走 defaultSkillId, 结果草原黏泥怪会放「旋风斩」、血牙狼自带冲锋被动--
338 * 魔物读起来像穿着玩家职业皮的假人, 第一章小怪也因此偏强。现在小怪默认无技能,
339 * Boss / 高级怪通过关卡蓝图的 `skillSkin` 显式挂上 (见 `enemySkillCatalog`)。
340 */
341 export function effectiveUnitDef(
342   u: UnitState,
343   defs: Record<UnitKind, UnitArchetypeDef>,
344 ): UnitDef {
345   const b = defs[u.defId];
346   const sk = u.faction === 'enemy'
347     ? u.battleSkill
348     : (u.battleSkill ?? skillDefForId(defaultSkillId(u.defId)));
349   const base = b.base;
350   const strike = b.strike;
```

```
351 const baseAtk = u.mercAtk ?? base.atk;
352 const baseSpd = u.mercSpd ?? base.spd;
353 const baseMove = u.mercMove ?? base.move;
354 const baseMaxHp = u.mercMaxHp ?? base.maxHp;
355 const baseRange = u.mercRange ?? strike.range;
356 const baseRanged = u.mercIsRanged ?? strike.isRanged;
357 const strikeTaunt = u.mercTaunt ?? strike.taunt;
358 const taunt = strikeTaunt || timedTauntActive(u);
359 const timedAtk = sumTimedAtkBonus(u);
360 const timedAtkDown = sumTimedAtkDown(u);
361 const timedSpd = sumTimedSpdBonus(u);
362 const timedSpdDown = sumTimedSpdDown(u);
363 return {
364   id: b.id,
365   name: b.name,
366   skill: sk,
367   tempSkill: u.tempSkill,
368   maxHp: baseMaxHp,
369   atk: Math.max(1, baseAtk + (u.bonusAtk ?? 0) + timedAtk - timedAtkDown),
370   spd: Math.max(1, baseSpd + (u.bonusSpd ?? 0) + timedSpd - timedSpdDown),
371   move: Math.max(1, baseMove + (u.bonusMove ?? 0)),
372   range: baseRange,
373   isRanged: baseRanged,
374   taunt,
375 };
376 }
377 import type {
378   BattleEvent,
379   BattleReport,
380   Faction,
381   UnitArchetypeDef,
382   UnitKind,
383   UnitState,
384   Vec2,
385 } from './types';
386 import { effectiveUnitDef } from './effectiveUnit';
387 import { canAttackFrom, chooseTurnAction, selectAttackTarget, type AiDifficulty } from './ai';
388 import { computeDamage, terrainAttackNote, terrainDefenseNote } from './damage';
389 import { POTION_DEFS } from '@data/potionCatalog';
390 import { getTerrainAt, type TerrainGrid } from './grid';
391 import { MAX_BATTLE_ROUNDS } from './constants';
392 import { cellsFromDist, reachableCells, shortestPath4 } from './path';
393 import {
394   castSkillManual,
395   skillAiming,
396   type SkillSlot,
397   trySkillAfterMove,
398   trySkillBeforeMove,
399   unitSkillSpec,
400   type SkillAiming,
```



```
401 } from './skills';
402 import { tickTimedBattleEffects } from './timedBattleEffects';
403 import { getTerrainSpec } from '@data/terrainSpec';
404 function key(p: Vec2): string {
405   return `${p.x},${p.y}`;
406 }
407 function cloneUnits(units: UnitState[]): UnitState[] {
408   return units.map((u) => ({
409     uid: u.uid,
410     defId: u.defId,
411     faction: u.faction,
412     hp: u.hp,
413     pos: { ...u.pos },
414     skillCd: u.skillCd ?? 0,
415     movedInTurn: false,
416     battleSkill: u.battleSkill,
417     tempSkill: u.tempSkill,
418     tempSkillCd: u.tempSkillCd ?? 0,
419     skillMods: u.skillMods ? [...u.skillMods] : undefined,
420     bonusAtk: u.bonusAtk,
421     bonusSpd: u.bonusSpd,
422     bonusMove: u.bonusMove,
423     rosterId: u.rosterId,
424     displayName: u.displayName,
425     boss: u.boss,
426     animSet: u.animSet,
427     mercMaxHp: u.mercMaxHp,
428     mercAtk: u.mercAtk,
429     mercSpd: u.mercSpd,
430     mercMove: u.mercMove,
431     mercRange: u.mercRange,
432     mercIsRanged: u.mercIsRanged,
433     mercTaunt: u.mercTaunt,
434     timedBattleEffects: u.timedBattleEffects?.map((e) => {
435       switch (e.kind) {
436         case 'taunt':
437           return { kind: 'taunt' as const, roundsLeft: e.roundsLeft };
438         case 'atkBonus':
439           return { kind: 'atkBonus' as const, addAtk: e.addAtk, roundsLeft: e.roundsLeft };
440         case 'atkDown':
441           return { kind: 'atkDown' as const, subAtk: e.subAtk, roundsLeft: e.roundsLeft };
442         case 'spdDown':
443           return { kind: 'spdDown' as const, subSpd: e.subSpd, roundsLeft: e.roundsLeft };
444         case 'spdBonus':
445           return { kind: 'spdBonus' as const, addSpd: e.addSpd, roundsLeft: e.roundsLeft };
446         case 'poison':
447           return { kind: 'poison' as const, dmgPerRound: e.dmgPerRound, roundsLeft: e.roundsLeft };
448       }
449     }),
450   }));
```

```

451 }
452 function buildBlocked(units: UnitState[], selfUid: string): Set<string> {
453     const s = new Set<string>();
454     for (const u of units) {
455         if (u.hp <= 0) continue;
456         if (u.uid === selfUid) continue;
457         s.add(key(u.pos));
458     }
459     return s;
460 }
461 function bySpeedOrder(units: UnitState[], defs: Record<UnitKind, UnitArchetypeDef>): UnitState[] {
462     return [...units]
463         .filter((u) => u.hp > 0)
464         .sort((a, b) => {
465             const sa = effectiveUnitDef(a, defs).spd;
466             const sb = effectiveUnitDef(b, defs).spd;
467             if (sb !== sa) return sb - sa;
468             return a.uid.localeCompare(b.uid);
469         });
470 }
471 function checkWinner(units: UnitState[]): 'player' | 'enemy' | null {
472     const p = units.some((u) => u.faction === 'player' && u.hp > 0);
473     const e = units.some((u) => u.faction === 'enemy' && u.hp > 0);
474     if (p && e) return null;
475     if (p) return 'player';
476     return 'enemy';
477 }
478 /**
479  * `manual`: 玩家单位逐个停下来等指令（走位 / 技能 / 普攻全由玩家决定）。
480  * `auto`: 全部交给程序决策代打，用于扫荡已通过的关卡。
481  */
482 export type BattleMode = 'manual' | 'auto';
483 export interface BattleSimOptions {
484     aiDifficulty?: AiDifficulty;
485     /** 缺省 `auto`，保持 `runBattle` 等纯模拟调用方的行为不变 */
486     mode?: BattleMode;
487     /**
488      * 敌人死亡时是否在死亡格掉药剂。只给无尽试炼开：
489      * 主线有商店买药，再在战场上掉会让补给点变得可有可无。
490      */
491     enableDrops?: boolean;
492     /** 掉落掷点，测试传入固定序列 */
493     dropRng?: () => number;
494     dropChance?: number;
495 }
496 /**
497  * 人工模式下当前单位的内部进度。
498  *
499  * 一个回合里**移动、技能、普攻各一次**，顺序不限。这里不采用「行动即结束回合」的经典
500  * 战棋规则，因为程序决策走的 `actTurn` 是技能和普攻都打的（见 `actTurn`）--限制成二选一，

```

```
501 * 人工输出就直接比自动低一截，扫荡会严格优于亲手打，那玩家没有理由认真玩。
502 * 规则对齐程序决策之后，人工模式相对自动的优势才落在「走位和时机」这件唯一该由人做的事上。
503 *
504 * `startPos` 只为撤销移动留着。撤销不需要整份快照：移动只改 `pos` 和 `movedInTurn`
505 * （地形每回合掉血在 `startRound` 结算，不在踏入时触发），复位这两个就是精确回滚。
506 */
507 interface MutablePending {
508   uid: string;
509   moved: boolean;
510   usedSkill: boolean;
511   attacked: boolean;
512   startPos: Vec2;
513 }
514 /** 一次 step 的结果：本步产生的事件 + 是否结束 */
515 export interface BattleStep {
516   events: BattleEvent[];
517   done: boolean;
518   winner: Faction | null;
519 }
520 /**
521 * 人工模式下，当前行动单位还剩哪些操作没用。
522 *
523 * `can*` 和 `did*` 都要给：光看 `canSkill: false` 分不清是「这回合已经放过了」
524 * 还是「冷却中 / 范围里没目标」。玩家看到一个灰掉的按钮，这两种原因要求的
525 * 下一步完全不同（前者是接着打普攻，后者是走位或换人），所以界面必须能区分。
526 */
527 /** 两个技能槽的固定顺序，UI 生成按钮和引擎枚举可放槽位都按它走 */
528 const SKILL_SLOTS: readonly SkillSlot[] = ['main', 'temp'];
529 export interface PendingTurn {
530   uid: string;
531   /** 还能移动（每回合一次；与技能/普攻顺序不限，和程序决策 `actTurn` 一致） */
532   canMove: boolean;
533   /**
534    * 还能放技能（任一槽）。
535    *
536    * 两个槽**共用**这一个额度：临时技能给的是「主技能在冷却时还有别的事可做」，
537    * 不是每回合多打一发。具体哪些槽此刻放得出来看 `castableSlots`。
538    */
539   canSkill: boolean;
540   /** 此刻放得出来的槽（主 / 临时），按钮按它来生成 */
541   castableSlots: SkillSlot[];
542   /** 还能普攻 */
543   canAttack: boolean;
544   /** 已经移动过、但还没出手，可以撤销回原位 */
545   canUndoMove: boolean;
546   didMove: boolean;
547   didSkill: boolean;
548   didAttack: boolean;
549 }
550 /**
```

```
551 * 逐步战斗模拟器。
552 *
553 * 两种推进方式，视图侧按`pending()`是否为空来分流：
554 * - `stepTurn()`：推进一个回合边界，或让一个**程序决策单位**打完整个回合（敌方 / 自动模式）。
555 * - `command*()`：人工模式下逐个动作推进当前玩家单位，每个都返回本次产生的事件。
556 *
557 * 两条路产出的事件格式完全一致，所以回放层只需要一套`playEvent`。
558 */
559 export interface BattleSim {
560   stepTurn(): BattleStep;
561   /** 人工模式下正在等指令的单位；null = 该调`stepTurn()`了 */
562   pending(): PendingTurn | null;
563   /** 从当前位置可移动到的格（不含脚下那格） */
564   legalMoveCells(uid: string): Vec2[];
565   /** 从当前位置能普攻到的敌人 uid */
566   legalAttackTargets(uid: string): string[];
567   /** 技能瞄准信息；null = 当前放不出来（冷却 / 范围内无目标 / 已用过） */
568   skillAiming(uid: string, slot?: SkillSlot): SkillAiming | null;
569   /** 移动到指定格（必须在`legalMoveCells`内，否则原地不动返回空事件） */
570   commandMove(uid: string, cell: Vec2): BattleStep;
571   /** 撤销本回合的移动（仅在还没出手时可用） */
572   commandUndoMove(uid: string): BattleStep;
573   /**
574    * 放技能。
575    * - 直线/范围确认：`aimCell`见`SkillAiming.aimCells`
576    * - 无需瞄准：两者都可省略
577    */
578   commandSkill(uid: string, targetUid?: string, slot?: SkillSlot, aimCell?: Vec2): BattleStep;
579   commandAttack(uid: string, targetUid: string): BattleStep;
580   /** 结束该单位回合（待机） */
581   commandWait(uid: string): BattleStep;
582   /** 使用药剂（立即生效，返回产生的事件供回放展示） */
583   usePotion(potionId: string): BattleEvent[];
584   /**
585    * 中途切换托管（自动 / 手动），返回**切换这一下**立即产生的事件供回放播出。
586    */
587   * 切到自动时，正在等指令的那个单位由程序决策接手打完（没用完的动作接着打，见`actTurn`
588   * 的 resume），所以会有事件；切回手动时只是不再代打**下一个**单位，事件为空。
589   * 已经结算完的动作不回滚--玩家从下一个该他动的单位开始接手。
590   */
591   setAuto(on: boolean): BattleStep;
592   /** 当前是不是程序决策代打玩家单位 */
593   isAuto(): boolean;
594   /**
595    * 跳过：一口气模拟到结束，返回完整战报（含之前已消费的事件）。
596    * 人工模式下也可用--剩下的交给程序决策代打，当前单位没用完的动作会被接着打完。
597    */
598   runToEnd(): BattleReport;
599   /** 当前存活单位（实时状态，供 UI 读取血量/冷却） */
600   getUnits(): UnitState[];
```

```
601  getUnit(uid: string): UnitState | undefined;
602  /**
603   * 本回合还没动到的单位 (uid) 。
604   * 当前正在行动的人已被弹出，不在这里--UI 要自己把 current 拼到队首。
605   */
606  roundOrder(): string[];
607  /**
608   * 出手顺序预览：当前行动者（可选）+ 本回合剩余 + 按速度预估的后续回合，
609   * 凑满 `limit` 个。顺序条要靠这条做跨回合规划，不能只看本回合尾巴。
610   */
611  upcomingOrder(limit: number, currentUid?: string | null): string[];
612  getRound(): number;
613  isDone(): boolean;
614  }
615  export function createBattleSim(
616    initialUnits: UnitState[],
617    terrain: TerrainGrid,
618    defs: Record<UnitKind, UnitArchetypeDef>,
619    opts: BattleSimOptions = {},
620  ): BattleSim {
621    const units = cloneUnits(initialUnits);
622    const aiDifficulty = opts.aiDifficulty ?? 'normal';
623    const mode: BattleMode = opts.mode ?? 'auto';
624    const enableDrops = opts.enableDrops ?? false;
625    const dropRng = opts.dropRng ?? Math.random;
626    const dropChance = opts.dropChance ?? 0.35;
627    const pickups: { pos: Vec2; potionId: string }[] = [];
628    const rolledDeaths = new Set<string>();
629    const DROP_POTIONS = ['heal', 'heal', 'heal', 'draught', 'slow'] as const;
630    const allEvents: BattleEvent[] = [];
631    /**
632     * 置位时玩家单位也由程序决策接手（开局选了自动，或中途切了托管 / 按了跳过）。
633     *
634     * 和 `mode` 分开：`mode` 是这一局的开场状态，`forceAuto` 是当下的实际状态。
635     * 合成一个变量的话「托管」就成了单向门--切过去之后没有东西记得原本该由谁操作。
636     */
637    let forceAuto = mode === 'auto';
638    /** 人工模式下当前停下来等指令的单位 */
639    let pendingTurn: MutablePending | null = null;
640    let rounds = 0;
641    let order: string[] = [];
642    let done = false;
643    let winner: Faction | null = null;
644    function attachDrops(events: BattleEvent[]): BattleEvent[] {
645      if (!enableDrops) return events;
646      const extra: BattleEvent[] = [];
647      for (const ev of events) {
648        if (ev.type !== 'death') continue;
649        if (rolledDeaths.has(ev.uid)) continue;
650        rolledDeaths.add(ev.uid);
```

```

651     const u = units.find((x) => x.uid === ev.uid);
652     if (!u || u.faction !== 'enemy') continue;
653     if (pickups.some((p) => p.pos.x === u.pos.x && p.pos.y === u.pos.y)) continue;
654     if (dropRng() >= dropChance) continue;
655     const potionId = DROP_POTIONS[Math.floor(dropRng() * DROP_POTIONS.length)];
656     pickups.push({ pos: { ...u.pos }, potionId });
657     extra.push({ type: 'drop', pos: { ...u.pos }, potionId });
658 }
659 return extra.length ? [...events, ...extra] : events;
660 }
661 function tryPickup(u: UnitState): BattleEvent[] {
662     const i = pickups.findIndex((p) => p.pos.x === u.pos.x && p.pos.y === u.pos.y);
663     if (i < 0) return [];
664     const drop = pickups.splice(i, 1)[0];
665     return [{ type: 'pickup', uid: u.uid, pos: { ...drop.pos }, potionId: drop.potionId }];
666 }
667 function finish(w: Faction, events: BattleEvent[]): BattleStep {
668     done = true;
669     winner = w;
670     pendingTurn = null;
671     const evs = attachDrops(events);
672     evs.push({ type: 'end', winner: w });
673     allEvents.push(...evs);
674     return { events: evs, done: true, winner: w };
675 }
676 /** 沿最短路走到 `to`，逐格发 moveStep 供回放做动画 */
677 function moveAlong(self: UnitState, to: Vec2): BattleEvent[] {
678     const path = shortestPath4(self.pos, to, buildBlocked(units, self.uid), terrain);
679     if (!path || path.length <= 1) return [];
680     const events: BattleEvent[] = [];
681     for (let i = 1; i < path.length; i++) {
682         const from = { ...self.pos };
683         const next = { ...path[i] };
684         self.pos = next;
685         self.movedInTurn = true;
686         events.push({ type: 'moveStep', uid: self.uid, from, to: next });
687     }
688     return events;
689 }
690 function liveUnit(uid: string): UnitState | null {
691     const u = units.find((x) => x.uid === uid);
692     return u && u.hp > 0 ? u : null;
693 }
694 function reachFrom(u: UnitState): Vec2[] {
695     const atkDef = effectiveUnitDef(u, defs);
696     const dist = reachableCells(u.pos, atkDef.move, buildBlocked(units, u.uid), terrain);
697     return cellsFromDist(u.pos, dist).filter((c) => !(c.x === u.pos.x && c.y === u.pos.y));
698 }
699 function attackTargetsFrom(u: UnitState): string[] {
700     const atkDef = effectiveUnitDef(u, defs);

```

```
701   return units
702   .filter((t) => t.hp > 0 && t.faction !== u.faction && canAttackFrom(atkDef, u.pos, t))
703   .map((t) => t.uid);
704 }
705 function startRound(): BattleStep {
706   rounds += 1;
707   const events: BattleEvent[] = [{ type: 'round', round: rounds }];
708   for (const t of tickTimedBattleEffects(units)) {
709     events.push({
710       type: 'dot',
711       uid: t.uid,
712       damage: t.damage,
713       hpLeft: t.hpLeft,
714       source: 'poison',
715     });
716     if (t.died) events.push({ type: 'death', uid: t.uid });
717   }
718   for (const u of units) {
719     if (u.hp <= 0) continue;
720     u.movedInTurn = false;
721     if (u.skillCd > 0) u.skillCd -= 1;
722     if ((u.tempSkillCd ?? 0) > 0) u.tempSkillCd = (u.tempSkillCd ?? 0) - 1;
723     const tSpec = getTerrainSpec(getTerrainAt(terrain, u.pos));
724     if (tSpec.dotPerRound > 0) {
725       u.hp -= tSpec.dotPerRound;
726       events.push({
727         type: 'dot',
728         uid: u.uid,
729         damage: tSpec.dotPerRound,
730         hpLeft: Math.max(0, u.hp),
731         source: 'terrain',
732       });
733       if (u.hp <= 0) {
734         events.push({ type: 'death', uid: u.uid });
735       }
736     }
737   }
738   order = bySpeedOrder(units, defs).map((u) => u.uid);
739   const evs = attachDrops(events);
740   const w = checkWinner(units);
741   if (w) return finish(w, evs);
742   allEvents.push(...evs);
743   return { events: evs, done: false, winner: null };
744 }
745 /**
746  * 程序决策打完一个单位的整个回合。
747  *
748  * `resume` 是人工模式下按了跳过时传进来的进度：该单位可能已经走过、或已经放过技能。
749  * 不传这个就只能二选一--要么让程序决策从头再走一遍（同一回合移动两次），要么直接丢掉
750  * 它剩下的动作。后者在按跳过的那一刻会白送一次攻击机会，而玩家通常正是因为
```

```
751  * 局势已定才按跳过的，凭空掉一刀会让结果和他的判断不符。
752  */
753  function actTurn(self: UnitState, resume?: MutablePending): BattleStep {
754    const events: BattleEvent[] = [];
755    const canSkill = !resume?.usedSkill;
756    const canMove = !resume?.moved;
757    const canAttack = !resume?.attacked;
758    if (resume?.moved) self.movedInTurn = true;
759    if (canSkill) {
760      events.push(...trySkillBeforeMove(self, defs, units, terrain));
761    }
762    let w = checkWinner(units);
763    if (w) return finish(w, events);
764    const choice = chooseTurnAction(
765      self, defs, units, terrain,
766      self.faction === 'enemy' ? aiDifficulty : 'normal',
767    );
768    const atkDef = effectiveUnitDef(self, defs);
769    if (canMove) {
770      const blockedReach = buildBlocked(units, self.uid);
771      const reachDist = reachableCells(self.pos, atkDef.move, blockedReach, terrain);
772      events.push({
773        type: 'moveRange',
774        uid: self.uid,
775        cells: cellsFromDist(self.pos, reachDist),
776      });
777    }
778    if (canMove && choice.moveTo) {
779      events.push(...moveAlong(self, choice.moveTo));
780    }
781    if (canSkill) {
782      events.push(...trySkillAfterMove(self, defs, units, terrain));
783    }
784    w = checkWinner(units);
785    if (w) return finish(w, events);
786    // 已经由玩家走过位了，就不能沿用 choice 里那个「配着别的落点」算出来的目标
787    const tgt = !canAttack
788      ? null
789      : canMove
790        ? choice.attackTarget
791        : selectAttackTarget(
792          atkDef,
793          self.pos,
794          units.filter((u) => u.faction !== self.faction),
795          defs,
796          self.faction === 'enemy' ? aiDifficulty : 'normal',
797        );
798    if (tgt && tgt.hp > 0) {
799      const tLive = units.find((u) => u.uid === tgt.uid);
800      // 目标可能已被技能击杀，或因位移脱战
```



```

801     if (tLive && tLive.hp > 0 && canAttackFrom(atkDef, self.pos, tLive)) {
802         events.push(...basicAttack(self, tLive));
803     }
804 }
805 w = checkWinner(units);
806 if (w) return finish(w, events);
807 const evs = attachDrops(events);
808 allEvents.push(...evs);
809 return { events: evs, done: false, winner: null };
810 }
811 /**
812  * 一次普攻的结算。程序决策回合和玩家指令共用，不要各写一份--
813  * 冲锋加成、地形归因这些散在两条路上迟早会分叉，而分叉的表现是
814  * 「同一招我打出 12、电脑打出 15」，玩家只会得出数值不可信的结论。
815  */
816 function basicAttack(self: UnitState, target: UnitState): BattleEvent[] {
817     const atkDef = effectiveUnitDef(self, defs);
818     const defT = effectiveUnitDef(target, defs);
819     let dmg = computeDamage(atkDef, defT, terrain, self.pos, target.pos);
820     const sk = atkDef.skill;
821     // 走 `unitSkillSpec` 而不是 `getSkillSpec`：冲锋的倍率也吃词条（「蓄势」「践地」），
822     // 读原始规格的话那两条选了等于没选，而表现只是普攻数字偏小，肉眼查不出来。
823     const chargeMul = sk ? unitSkillSpec(self, sk.id)?.passiveBasicAttackMulIfMoved : undefined;
824     const charged = Boolean(chargeMul) && self.movedInTurn;
825     if (chargeMul && charged) {
826         dmg = Math.max(1, Math.floor(dmg * chargeMul));
827     }
828     target.hp -= dmg;
829     const events: BattleEvent[] = [{
830         type: 'attack',
831         attacker: self.uid,
832         target: target.uid,
833         damage: dmg,
834         hpLeft: Math.max(0, target.hp),
835         // 冲锋和地形一样，改了这一下的伤害就得说出来。骑兵的全部身份就是「先动再打更疼」，
836         // 而在此之前它只体现为一个玩家无从比较的数字，等于这个被动不存在。
837         attackLabel: charged ? '冲锋' : '普攻',
838         charged: charged || undefined,
839         // 地形改了这一下的伤害就说出来。地形之前「没用」很大一部分是因为它从不出声：
840         // 高地 +25% 只体现为一个玩家无从比较的数字。
841         atkTerrainNote: terrainAttackNote(terrain, self.pos) ?? undefined,
842         defTerrainNote: terrainDefenseNote(terrain, target.pos) ?? undefined,
843     }];
844     if (target.hp <= 0) events.push({ type: 'death', uid: target.uid });
845     return events;
846 }
847 function stepTurn(): BattleStep {
848     if (done) return { events: [], done: true, winner };
849     // 还有人在等指令时不许推进，否则玩家那个单位会被跳过
850     if (pendingTurn) return { events: [], done: false, winner: null };

```

```
851 // 回合边界
852 if (order.length === 0) {
853     const w0 = checkWinner(units);
854     if (w0) return finish(w0, []);
855     if (rounds >= MAX_BATTLE_ROUNDS) return finish('enemy', []);
856     return startRound();
857 }
858 // 弹出下一个存活行动者
859 while (order.length > 0) {
860     const uid = order.shift();
861     const self = units.find((u) => u.uid === uid);
862     if (!self || self.hp <= 0) continue;
863     const turnStart: BattleEvent = { type: 'turnStart', uid: self.uid, faction: self.faction };
864     if (self.faction === 'player' && !forceAuto) {
865         pendingTurn = {
866             uid: self.uid,
867             moved: false,
868             usedSkill: false,
869             attacked: false,
870             startPos: { ...self.pos },
871         };
872         allEvents.push(turnStart);
873         return { events: [turnStart], done: false, winner: null };
874     }
875     const step = actTurn(self);
876     return { ...step, events: [turnStart, ...step.events] };
877 }
878 // 本回合行动者全部阵亡 -> 直接进入下一回合边界
879 return stepTurn();
880 }
881 // ===== 人工模式：逐指令推进 =====
882 /** 当前等指令的单位；顺带处理它已经死了的情况（被反击/地形打死不会发生，但别留坑） */
883 function activePending(uid: string): { p: MutablePending; u: UnitState } | null {
884     if (done || !pendingTurn || pendingTurn.uid !== uid) return null;
885     const u = liveUnit(uid);
886     if (!u) return null;
887     return { p: pendingTurn, u };
888 }
889 function pendingView(): PendingTurn | null {
890     if (!pendingTurn) return null;
891     const u = liveUnit(pendingTurn.uid);
892     if (!u) return null;
893     const acted = pendingTurn.usedSkill || pendingTurn.attacked;
894     const castable = castableSlotsFor(u);
895     return {
896         uid: pendingTurn.uid,
897         // 移动与出手顺序不限：程序决策是先技能再走位再普攻的（见`actTurn`），
898         // 人工若「先放技能就锁死移动」，玩家永远学不会那套起手，也会白白丢掉走位。
899         // 撤招只在「走过且还没出手」时开放-出手后再撤等于改写已经结算的伤害。
900         canMove: !pendingTurn.moved,
```

```
901     canSkill: castable.length > 0,
902     castableSlots: castable,
903     canAttack: !pendingTurn.attacked && attackTargetsFrom(u).length > 0,
904     canUndoMove: pendingTurn.moved && !acted,
905     didMove: pendingTurn.moved,
906     didSkill: pendingTurn.usedSkill,
907     didAttack: pendingTurn.attacked,
908   };
909 }
910 function skillAimingFor(u: UnitState, slot: SkillSlot = 'main'): SkillAiming | null {
911   if (pendingTurn?.uid === u.uid && pendingTurn.usedSkill) return null;
912   return skillAiming(u, defs, units, terrain, slot);
913 }
914 function castableSlotsFor(u: UnitState): SkillSlot[] {
915   return SKILL_SLOTS.filter((slot) => skillAimingFor(u, slot) !== null);
916 }
917 /**
918  * 一个动作结算完之后：该结束回合就结束。
919  *
920  * 「无事可做就自动收尾」省掉了绝大多数回合里那一下多余的「结束」点击--
921  * 常见回合就是走一步、砍一刀，技能在冷却，此时再要求玩家确认一次纯属噪音。
922  * 但只要还有技能能放，就一定停下来等：替玩家决定放弃一次技能，比多一次点击槽得多。
923  */
924 function settleAfterAction(events: BattleEvent[]): BattleStep {
925   const evs = attachDrops(events);
926   const w = checkWinner(units);
927   if (w) return finish(w, evs);
928   const v = pendingView();
929   // 还有移动/出手额度，或还能撤销走位时，都停下来等。
930   // 走完若立刻收尾，玩家来不及点撤销；先技能后走位也不能出手完就掐掉移动。
931   if (!v || (!v.canMove && !v.canSkill && !v.canAttack && !v.canUndoMove)) {
932     pendingTurn = null;
933   }
934   allEvents.push(...evs);
935   return { events: evs, done: false, winner: null };
936 }
937 function noop(): BattleStep {
938   return { events: [], done: false, winner: null };
939 }
940 function commandMove(uid: string, cell: Vec2): BattleStep {
941   const cur = activePending(uid);
942   if (!cur) return noop();
943   const v = pendingView();
944   if (!v?.canMove) return noop();
945   if (!reachFrom(cur.u).some((c) => c.x === cell.x && c.y === cell.y)) return noop();
946   const events = moveAlong(cur.u, cell);
947   if (events.length === 0) return noop();
948   cur.p.moved = true;
949   // 走完若技能/普攻也没了，settle 会自动收尾；还有出手额度则继续等
950   return settleAfterAction(events);
```

```
951 }
952 function commandUndoMove(uid: string): BattleStep {
953     const cur = activePending(uid);
954     if (!cur) return noop();
955     const v = pendingView();
956     if (!v?.canUndoMove) return noop();
957     const from = { ...cur.u.pos };
958     cur.u.pos = { ...cur.p.startPos };
959     cur.u.movedInTurn = false;
960     cur.p.moved = false;
961     // 用一步 moveStep 直接跳回原位，回放层按普通移动播即可
962     const events: BattleEvent[] = [
963         { type: 'moveStep', uid, from, to: { ...cur.p.startPos } },
964     ];
965     allEvents.push(...events);
966     return { events, done: false, winner: null };
967 }
968 function commandSkill(
969     uid: string,
970     targetUid?: string,
971     slot: SkillSlot = 'main',
972     aimCell?: Vec2,
973 ): BattleStep {
974     const cur = activePending(uid);
975     if (!cur) return noop();
976     const v = pendingView();
977     if (!v?.castableSlots.includes(slot)) return noop();
978     const events = castSkillManual(cur.u, defs, units, terrain, targetUid, slot, aimCell);
979     if (events.length === 0) return noop();
980     cur.p.usedSkill = true;
981     return settleAfterAction(events);
982 }
983 function commandAttack(uid: string, targetUid: string): BattleStep {
984     const cur = activePending(uid);
985     if (!cur) return noop();
986     const v = pendingView();
987     if (!v?.canAttack) return noop();
988     if (!attackTargetsFrom(cur.u).includes(targetUid)) return noop();
989     const target = liveUnit(targetUid);
990     if (!target) return noop();
991     cur.p.attacked = true;
992     return settleAfterAction(basicAttack(cur.u, target));
993 }
994 function commandWait(uid: string): BattleStep {
995     const cur = activePending(uid);
996     if (!cur) return noop();
997     // 走到掉落格上再待机才捡起来：只路过不捡，否则走位会被「顺手拾取」绑死
998     const events = tryPickup(cur.u);
999     pendingTurn = null;
1000     allEvents.push(...events);
```

```
1001   return { events, done: false, winner: null };
1002 }
1003 function usePotion(potionId: string): BattleEvent[] {
1004   if (done) return [];
1005   const def = POTION_DEFS[potionId];
1006   if (!def) return [];
1007   const events: BattleEvent[] = [{ type: 'potion', potionId, name: def.name }];
1008   const eff = def.effect;
1009   if (eff.kind === 'healAllies') {
1010     for (const u of units) {
1011       if (u.faction !== 'player' || u.hp <= 0) continue;
1012       const maxHp = effectiveUnitDef(u, defs).maxHp;
1013       const amount = Math.max(1, Math.floor(maxHp * eff.ratio));
1014       const healed = Math.min(maxHp, u.hp + amount) - u.hp;
1015       if (healed <= 0) continue;
1016       u.hp += healed;
1017       events.push({ type: 'heal', target: u.uid, amount: healed, hpLeft: u.hp });
1018     }
1019   } else if (eff.kind === 'atkBuffAllies') {
1020     for (const u of units) {
1021       if (u.faction !== 'player' || u.hp <= 0) continue;
1022       const baseAtk = effectiveUnitDef(u, defs).atk;
1023       const addAtk = Math.max(1, Math.floor(baseAtk * eff.atkRatio));
1024       const list = u.timedBattleEffects ?? [];
1025       // +1: tick 在回合开始统一 -1, 保证 buff 覆盖 N 个完整回合
1026       list.push({ kind: 'atkBonus', addAtk, roundsLeft: eff.rounds + 1 });
1027       u.timedBattleEffects = list;
1028       events.push({ type: 'statusNote', target: u.uid, text: `攻+${addAtk}`, tone: 'buff' });
1029     }
1030   } else if (eff.kind === 'slowFoes') {
1031     for (const u of units) {
1032       if (u.faction !== 'enemy' || u.hp <= 0) continue;
1033       const list: typeof u.timedBattleEffects =
1034         (u.timedBattleEffects ?? []).filter((x) => x.kind !== 'spdDown');
1035       list.push({ kind: 'spdDown', subSpd: eff.subSpd, roundsLeft: eff.rounds + 1 });
1036       u.timedBattleEffects = list;
1037       events.push({
1038         type: 'statusNote',
1039         target: u.uid,
1040         text: `速-${eff.subSpd}`,
1041         tone: 'debuff',
1042       });
1043     }
1044   }
1045   allEvents.push(...events);
1046   return events;
1047 }
1048 function setAuto(on: boolean): BattleStep {
1049   forceAuto = on;
1050   const idle: BattleStep = { events: [], done, winner };
1051 }
```

```
1051 // 切回手动不需要动当前状态：下一次 stepTurn 自然会停下来等指令。
1052 if (!on || done || !pendingTurn) return idle;
1053 // 当前单位如果已经走了一半，把剩下的动作接着打完（见 actTurn 的 resume）
1054 const resume = pendingTurn;
1055 pendingTurn = null;
1056 const u = liveUnit(resume.uid);
1057 return u ? actTurn(u, resume) : idle;
1058 }
1059 function runToEnd(): BattleReport {
1060   setAuto(true);
1061   while (!done) stepTurn();
1062   return { events: allEvents, winner: winner ?? 'enemy', rounds };
1063 }
1064 return {
1065   stepTurn,
1066   pending: pendingView,
1067   legalMoveCells: (uid) => {
1068     const u = liveUnit(uid);
1069     return u ? reachFrom(u) : [];
1070   },
1071   legalAttackTargets: (uid) => {
1072     const u = liveUnit(uid);
1073     return u ? attackTargetsFrom(u) : [];
1074   },
1075   skillAiming: (uid, slot) => {
1076     const u = liveUnit(uid);
1077     return u ? skillAimingFor(u, slot ?? 'main') : null;
1078   },
1079   commandMove,
1080   commandUndoMove,
1081   commandSkill,
1082   commandAttack,
1083   commandWait,
1084   usePotion,
1085   setAuto,
1086   isAuto: () => forceAuto,
1087   runToEnd,
1088   getUnits: () => units,
1089   getUnit: (uid) => units.find((u) => u.uid === uid),
1090   roundOrder: () => [...order],
1091   upcomingOrder: (limit, currentUid) => {
1092     const alive = units.filter((u) => u.hp > 0);
1093     const aliveIds = new Set(alive.map((u) => u.uid));
1094     const out: string[] = [];
1095     if (currentUid && aliveIds.has(currentUid)) out.push(currentUid);
1096     for (const uid of order) {
1097       if (out.length >= limit) break;
1098       if (aliveIds.has(uid) && uid !== currentUid) out.push(uid);
1099     }
1100     // 本回合队列耗尽后，用当前速度序循环预估下一回合、下下回合.....
```

```

1101   while (out.length < limit && alive.length > 0) {
1102       const next = bySpeedOrder(alive, defs).map((u) => u.uid);
1103       let added = 0;
1104       for (const uid of next) {
1105           if (out.length >= limit) break;
1106           out.push(uid);
1107           added++;
1108       }
1109       if (added === 0) break;
1110   }
1111   return out;
1112 },
1113 getRound: () => rounds,
1114 isDone: () => done,
1115 };
1116 }
1117 /** 一次性模拟到结束（全自动）；「跳过回放」与单测复用 */
1118 export function runBattle(
1119   initialUnits: UnitState[],
1120   terrain: TerrainGrid,
1121   defs: Record<UnitKind, UnitArchetypeDef>,
1122   aiDifficulty: AiDifficulty = 'normal',
1123 ): BattleReport {
1124   const sim = createBattleSim(initialUnits, terrain, defs, { aiDifficulty, mode: 'auto' });
1125   return sim.runToEnd();
1126 }
1127 import type { TerrainId, Vec2 } from './types';
1128 export type TerrainGrid = TerrainId[][];
1129 export function gridSize(g: TerrainGrid): { w: number; h: number } {
1130   const h = g.length;
1131   const w = h > 0 ? (g[0]?.length ?? 0) : 0;
1132   return { w, h };
1133 }
1134 export function inBounds(p: Vec2, g: TerrainGrid): boolean {
1135   const { w, h } = gridSize(g);
1136   if (w <= 0 || h <= 0) return false;
1137   return p.x >= 0 && p.x < w && p.y >= 0 && p.y < h;
1138 }
1139 export function manhattan(a: Vec2, b: Vec2): number {
1140   return Math.abs(a.x - b.x) + Math.abs(a.y - b.y);
1141 }
1142 export function getTerrainAt(grid: TerrainGrid, p: Vec2): TerrainId {
1143   if (!inBounds(p, grid)) return 'plain';
1144   return grid[p.y][p.x] ?? 'plain';
1145 }
1146 /** 4 邻格（受当前关卡地形图尺寸约束） */
1147 export function neighbors4(p: Vec2, g: TerrainGrid): Vec2[] {
1148   return [
1149     { x: p.x + 1, y: p.y },
1150     { x: p.x - 1, y: p.y },

```

```
1151   { x: p.x, y: p.y + 1 },
1152   { x: p.x, y: p.y - 1 },
1153 ].filter((n) => inBounds(n, g));
1154 }
1155 /** 生成 `width × height` 全平原地形（每关棋盘尺寸由 `terrain` 矩阵决定） */
1156 export function emptyTerrain(width: number, height: number): TerrainGrid {
1157   return Array.from({ length: height }, () =>
1158     Array.from({ length: width }, (): TerrainId => 'plain'),
1159   );
1160 }
1161 /** 将玩家布阵阶段放置的地形叠到关卡底图上（浅拷贝行） */
1162 export function mergeTerrainOverlay(base: TerrainGrid, overlay: { x: number; y: number; terrain: ...
1163   const g = base.map((row) => [...row]);
1164   for (const c of overlay) {
1165     if (g[c.y]?[c.x] !== undefined) g[c.y][c.x] = c.terrain;
1166   }
1167   return g;
1168 }
1169 export type {
1170   BattleEvent,
1171   BattleReport,
1172   Faction,
1173   TerrainId,
1174   TimedBattleEffect,
1175   UnitDef,
1176   UnitKind,
1177   UnitState,
1178   Vec2,
1179 } from './types';
1180 export { MAX_BATTLE_ROUNDS, playerDeployRowRange, COUNTER_STRONG, COUNTER_WEAK } from './constants';
1181 export { runBattle } from './engine';
1182 export { effectiveUnitDef } from './effectiveUnit';
1183 export type { SkillDamageContext } from './skillDamage';
1184 export {
1185   computeSkillHitDamage,
1186   computeSkillHitDamageWithSpec,
1187   registerSkillDamageCalculator,
1188   unregisterSkillDamageCalculator,
1189 } from './skillDamage';
1190 export type { SkillDamageSpec } from '@data/skillCatalog';
1191 export { emptyTerrain, gridSize, inBounds, mergeTerrainOverlay, neighbors4 } from './grid';
1192 export type { TerrainGrid } from './grid';
1193 import type { TerrainGrid } from './grid';
1194 import { getTerrainAt, neighbors4 } from './grid';
1195 import type { Vec2 } from './types';
1196 import { getTerrainSpec } from '@data/terrainSpec';
1197 function key(p: Vec2): string {
1198   return `${p.x},${p.y}`;
1199 }
1200 /**
```



```
1201 * BFS with per-cell movement cost from TerrainSpec.
1202 * Impassable terrain (moveCost === Infinity) is never entered.
1203 */
1204 export function reachableCells(
1205   start: Vec2,
1206   moveBudget: number,
1207   blocked: Set<string>,
1208   terrain: TerrainGrid,
1209 ): Map<string, number> {
1210   const dist = new Map<string, number>();
1211   const q: Vec2[] = [start];
1212   dist.set(key(start), 0);
1213   let qi = 0;
1214   while (qi < q.length) {
1215     const p = q[qi++];
1216     const d = dist.get(key(p));
1217     for (const n of neighbors4(p, terrain)) {
1218       const nk = key(n);
1219       if (nk === key(start)) continue;
1220       if (blocked.has(nk)) continue;
1221       const tSpec = getTerrainSpec(getTerrainAt(terrain, n));
1222       if (tSpec.moveCost >= Infinity) continue;
1223       const nd = d + tSpec.moveCost;
1224       if (nd > moveBudget) continue;
1225       if (!dist.has(nk) || nd < dist.get(nk)!) {
1226         dist.set(nk, nd);
1227         q.push(n);
1228       }
1229     }
1230   }
1231   return dist;
1232 }
1233 /** All reachable cells (from BFS dist map) as Vec2[]. */
1234 export function cellsFromDist(_unusedStart: Vec2, dist: Map<string, number>): Vec2[] {
1235   const out: Vec2[] = [];
1236   for (const [k] of dist) {
1237     const [xs, ys] = k.split(',');
1238     out.push({ x: Number(xs), y: Number(ys) });
1239   }
1240   return out;
1241 }
1242 /** Shortest path (4-dir) respecting terrain movement cost. */
1243 export function shortestPath4(
1244   from: Vec2,
1245   to: Vec2,
1246   blocked: Set<string>,
1247   terrain: TerrainGrid,
1248 ): Vec2[] | null {
1249   const fk = key(from);
1250   const tk = key(to);
```

```
1251   if (fk === tk) return [from];
1252   if (blocked.has(tk)) return null;
1253   const tDest = getTerrainSpec(getTerrainAt(terrain, to));
1254   if (tDest.moveCost >= Infinity) return null;
1255   const dist = new Map<string, number>();
1256   const parent = new Map<string, string>();
1257   const q: Vec2[] = [from];
1258   dist.set(fk, 0);
1259   parent.set(fk, "");
1260   let qi = 0;
1261   while (qi < q.length) {
1262     const p = q[qi++];
1263     const pk = key(p);
1264     if (pk === tk) {
1265       const out: Vec2[] = [];
1266       let cur: string | undefined = tk;
1267       while (cur && cur !== fk) {
1268         const [xa, ya] = cur.split(',');
1269         out.push({ x: Number(xa), y: Number(ya) });
1270         cur = parent.get(cur);
1271       }
1272       out.push({ ...from });
1273       out.reverse();
1274       return out;
1275     }
1276     const d = dist.get(pk)!;
1277     for (const n of neighbors4(p, terrain)) {
1278       const nk = key(n);
1279       if (blocked.has(nk)) continue;
1280       const tSpec = getTerrainSpec(getTerrainAt(terrain, n));
1281       if (tSpec.moveCost >= Infinity) continue;
1282       const nd = d + tSpec.moveCost;
1283       if (!dist.has(nk) || nd < dist.get(nk)!) {
1284         dist.set(nk, nd);
1285         parent.set(nk, pk);
1286         q.push(n);
1287       }
1288     }
1289   }
1290   return null;
1291 }
1292 import type { SkillDamageSpec, SkillSpec } from '@data/skillCatalog';
1293 import { computeDamage, counterMultiplier, terrainAttackMul, terrainDefenseMul } from '../damage';
1294 import type { UnitDef } from '../types';
1295 import type { SkillDamageContext } from './context';
1296 function atkDefScaled(base: UnitDef, atk: number): UnitDef {
1297   return { ...base, atk };
1298 }
1299 function clampDamage(n: number): number {
1300   return Math.max(1, Math.floor(n));
```

```
1301 }
1302 /** `custom` 用: `registerSkillDamageCalculator(id, fn)` */
1303 const customCalculators = new Map<
1304   string,
1305   (ctx: SkillDamageContext, params: Record<string, number> | undefined) => number
1306 >();
1307 /**
1308  * 注册自定义伤害公式 (`damage: { kind: 'custom', id, params }`) 。
1309  * `fn` 返回整数伤害, 最终统一钳到  $\geq 1$ 。
1310  */
1311 export function registerSkillDamageCalculator(
1312   id: string,
1313   fn: (ctx: SkillDamageContext, params: Record<string, number> | undefined) => number,
1314 ): void {
1315   customCalculators.set(id, fn);
1316 }
1317 export function unregisterSkillDamageCalculator(id: string): void {
1318   customCalculators.delete(id);
1319 }
1320 function scaledStandard(ctx: SkillDamageContext, atkMul: number): number {
1321   if (atkMul <= 0) return 0;
1322   const atk = Math.max(1, Math.floor(ctx.casterDef.atk * atkMul));
1323   const sad = atkDefScaled(ctx.casterDef, atk);
1324   return clampDamage(computeDamage(sad, ctx.targetDef, ctx.terrain, ctx.self.pos, ctx.target.pos));
1325 }
1326 /**
1327  * `applyTerrain` 同时管**两头**的地形: 施法者站的格 (攻击加成) 和目标站的格 (减伤) 。
1328  * 只吃一头会让「站进森林」对普攻有效、对技能无效, 玩家学不会这条规则。
1329  */
1330 function terrainMul(ctx: SkillDamageContext): number {
1331   return terrainAttackMul(ctx.terrain, ctx.self.pos) * terrainDefenseMul(ctx.terrain, ctx.target....
1332 }
1333 function flatDamage(
1334   ctx: SkillDamageContext,
1335   amount: number,
1336   applyCounter: boolean,
1337   applyTerrain: boolean,
1338 ): number {
1339   let n = amount;
1340   if (applyCounter) n *= counterMultiplier(ctx.casterDef.id, ctx.targetDef.id);
1341   if (applyTerrain) n *= terrainMul(ctx);
1342   return clampDamage(n);
1343 }
1344 function percentTargetMaxHp(
1345   ctx: SkillDamageContext,
1346   ratio: number,
1347   applyCounter: boolean,
1348   applyTerrain: boolean,
1349 ): number {
1350   let n = ctx.targetDef.maxHp * ratio;
```

```
1351   if (applyCounter) n *= counterMultiplier(ctx.casterDef.id, ctx.targetDef.id);
1352   if (applyTerrain) n *= terrainMul(ctx);
1353   return clampDamage(n);
1354 }
1355 /**
1356  * 「处决」类词条的倍率：目标残血时才生效。
1357  *
1358  * 读 `target.hp` 而不是词条侧预判，因为同一次 AoE 里前面的目标已经掉过血了--
1359  * 谁进了处决线只有结算这一刻知道。
1360  */
1361 function executeMul(ctx: SkillDamageContext): number {
1362   const ex = ctx.spec.executeBonus;
1363   if (!ex) return 1;
1364   return isExecuting(ctx.spec, ctx.target.hp, ctx.targetDef.maxHp) ? ex.mul : 1;
1365 }
1366 /**
1367  * 这一击有没有踩到处决线。结算和飘字问的是同一个函数，两边算法分家的话
1368  * 会出现「飘了处决但伤害没涨」这种查不出来的对不上。
1369  *
1370  * 必须在扣血**之前**问：处决读的是命中那一刻的血量。
1371  */
1372 export function isExecuting(spec: SkillSpec, targetHp: number, targetMaxHp: number): boolean {
1373   const ex = spec.executeBonus;
1374   if (!ex || targetMaxHp <= 0) return false;
1375   return targetHp / targetMaxHp < ex.belowHpRatio;
1376 }
1377 /**
1378  * 对单个目标结算技能伤害（不含治疗等；`none` 为 0）。
1379  * 内置种类在 `SkillDamageSpec`；扩展用 `custom` + `registerSkillDamageCalculator`。
1380  */
1381 export function computeSkillHitDamage(ctx: SkillDamageContext): number {
1382   const base = dispatchDamage(ctx.spec.damage, ctx);
1383   if (base <= 0) return base;
1384   const mul = executeMul(ctx);
1385   return mul === 1 ? base : clampDamage(base * mul);
1386 }
1387 function dispatchDamage(d: SkillDamageSpec, ctx: SkillDamageContext): number {
1388   switch (d.kind) {
1389     case 'none':
1390       return 0;
1391     case 'scaledAtk':
1392       return scaledStandard(ctx, d.atkMul);
1393     case 'flat':
1394       return flatDamage(ctx, d.amount, d.applyCounter !== false, d.applyTerrain !== false);
1395     case 'percentTargetMaxHp':
1396       return percentTargetMaxHp(ctx, d.ratio, d.applyCounter !== false, d.applyTerrain !== false);
1397     case 'custom': {
1398       const fn = customCalculators.get(d.id);
1399       if (!fn) {
1400         throw new Error(`[skillDamage] unknown custom calculator id: "${d.id}" (skill ${ctx.spec....`
```

```
1401     }
1402     return clampDamage(fn(ctx, d.params));
1403 }
1404 }
1405 }
1406 /** 供测试或工具：对上下文用另一套 `damage` 试算 */
1407 export function computeSkillHitDamageWithSpec(
1408     ctx: SkillDamageContext,
1409     damage: SkillDamageSpec,
1410 ): number {
1411     return dispatchDamage(damage, ctx);
1412 }
1413 import type { UnitArchetypeDef, UnitDef, UnitKind, UnitState } from '../types';
1414 import type { SkillSpec } from '@data/skillCatalog';
1415 import type { TerrainGrid } from '../grid';
1416 /** 计算「对某一目标」的技能伤害时传入的上下文（多目标技能会多次调用） */
1417 export interface SkillDamageContext {
1418     self: UnitState;
1419     target: UnitState;
1420     /** 施法者本场合并面板 `effectiveUnitDef(self)` */
1421     casterDef: UnitDef;
1422     /** 目标本场合并面板 */
1423     targetDef: UnitDef;
1424     spec: SkillSpec;
1425     terrain: TerrainGrid;
1426     defs: Record<UnitKind, UnitArchetypeDef>;
1427 }
1428 export type { SkillDamageContext } from './context';
1429 export {
1430     computeSkillHitDamage,
1431     computeSkillHitDamageWithSpec,
1432     isExecuting,
1433     registerSkillDamageCalculator,
1434     unregisterSkillDamageCalculator,
1435 } from './computeSkillHitDamage';
1436 import type {
1437     BattleEvent,
1438     SkillDef,
1439     SkillHit,
1440     UnitArchetypeDef,
1441     UnitDef,
1442     UnitKind,
1443     UnitState,
1444     Vec2,
1445 } from '../types';
1446 import { effectiveUnitDef } from './effectiveUnit';
1447 import { terrainAttackNote, terrainDefenseNote } from './damage';
1448 import { computeSkillHitDamage, isExecuting } from './skillDamage';
1449 import {
1450     applySkillCastAllyEffects,
```

```
1451   applySkillCastFoeEffects,
1452   applySkillCastSelfEffects,
1453 } from './timedBattleEffects';
1454 import { canProfessionEquipSkill, getSkillSpec, type SkillSpec } from '@data/skillCatalog';
1455 import { effectiveSkillSpec } from '@data/skillModCatalog';
1456 import { gridSize, inBounds, manhattan, neighbors4, type TerrainGrid } from './grid';
1457 const RAY_DIRS: Vec2[] = [
1458   { x: 1, y: 0 },
1459   { x: -1, y: 0 },
1460   { x: 0, y: 1 },
1461   { x: 0, y: -1 },
1462 ];
1463 function livingFoes(self: UnitState, units: UnitState[]): UnitState[] {
1464   return units.filter((u) => u.hp > 0 && u.faction !== self.faction);
1465 }
1466 function livingAllies(self: UnitState, units: UnitState[]): UnitState[] {
1467   return units.filter((u) => u.hp > 0 && u.faction === self.faction);
1468 }
1469 function pushDeathIfNeeded(events: BattleEvent[], t: UnitState): void {
1470   if (t.hp <= 0) events.push({ type: 'death', uid: t.uid });
1471 }
1472 function enemiesOnRay(
1473   self: UnitState,
1474   from: Vec2,
1475   dir: Vec2,
1476   units: UnitState[],
1477   terrain: TerrainGrid,
1478 ): UnitState[] {
1479   const out: UnitState[] = [];
1480   let p = { x: from.x + dir.x, y: from.y + dir.y };
1481   while (inBounds(p, terrain)) {
1482     const occ = units.find((u) => u.hp > 0 && u.pos.x === p.x && u.pos.y === p.y);
1483     if (occ && occ.faction !== self.faction) out.push(occ);
1484     p = { x: p.x + dir.x, y: p.y + dir.y };
1485   }
1486   return out;
1487 }
1488 function rayCellsFrom(from: Vec2, dir: Vec2, terrain: TerrainGrid): Vec2[] {
1489   const cells: Vec2[] = [];
1490   let p = { x: from.x + dir.x, y: from.y + dir.y };
1491   while (inBounds(p, terrain)) {
1492     cells.push({ ...p });
1493     p = { x: p.x + dir.x, y: p.y + dir.y };
1494   }
1495   return cells;
1496 }
1497 function bestLinePick(
1498   self: UnitState,
1499   from: Vec2,
1500   units: UnitState[],
```

```
18558  /**
18559  * 显示冷却剩余回合。布阵页预览敌人时关掉：那时候还没开打，
18560  * 写「剩 0 回合」是在回答一个玩家没问的问题。
18561  */
18562  showCooldown: boolean;
18563  }
18564  /**
18565  * 战场单位（含布阵页的敌人预览）。
18566  *
18567  * 数值一律走 `effectiveUnitDef`--它是战斗结算读的同一个函数，
18568  * 所以面板上的攻击力就是这一刻真会打出去的攻击力，含限时增益减益。
18569  * 自己抄一遍加减法的话，加了新 buff 种类时面板会悄悄开始说谎。
18570  */
18571  export function battleUnitInfoModel(u: UnitState, opts: BattleUnitInfoOptions): UnitInfoModel {
18572    const ed = effectiveUnitDef(u, UNIT_DEFS);
18573    const kindName = UNIT_DEFS[u.defId].name;
18574    const skills: UnitInfoSkillSection[] = [];
18575    const cdNote = (left: number): string | undefined =>
18576      opts.showCooldown && left > 0 ? `（剩 ${left}）` : undefined;
18577    if (ed.skill) {
18578      const s = mainSection(ed.skill.id, u.skillMods, cdNote(u.skillCd));
18579      if (s) {
18580        // 敌方技能皮肤覆写展示名/图标；结算仍认 ed.skill.id -> SkillSpec
18581        if (u.battleSkill?.name) s.name = u.battleSkill.name;
18582        if (u.battleSkill?.iconKey) s.iconKey = u.battleSkill.iconKey;
18583        skills.push(s);
18584      }
18585    }
18586    if (u.tempSkill) {
18587      const s = tempSection(u.tempSkill.id, cdNote(u.tempSkillCd ?? 0));
18588      if (s) skills.push(s);
18589    }
18590    const statuses = (u.timedBattleEffects ?? []).
18591      .filter((e) => e.roundsLeft > 0)
18592      .map(describeTimedEffect);
18593    // 敌方副标题写「定位」而不是兵种名：草原杂兵的 defId 还是 sword/bow，
18594    // 但外观和名字已经是魔物了，面板上冒出「剑士」等于把实现细节漏给玩家。
18595    // 我方相反，职业名正是玩家认人的方式。
18596    const role = ed.isRanged ? '远程' : '近战';
18597    const subtitle = u.faction === 'enemy'
18598      ? `${u.boss ? '首领' : '敌方'} · ${role}`
18599      : kindName;
18600    return {
18601      name: u.displayName ?? kindName,
18602      subtitle,
18603      createPortrait: () => createUnitToken(u.animSet ?? u.defId, u.faction, 48),
18604      stats: [
18605        { label: '生命', value: `${Math.max(0, u.hp)}/${ed.maxHp}` },
18606        { label: '攻击', value: `${ed.atk}` },
18607        { label: '速度', value: `${ed.spd}` },
```

```
18608   { label: '移动', value: `${ed.move}` },
18609 ],
18610 strikeTitle: '普通攻击',
18611 strike: [
18612   { label: '射程', value: `${ed.range}` },
18613   { label: '类型', value: ed.isRanged ? '远程' : '近战' },
18614   { label: '嘲讽', value: ed.taunt ? '是' : '否' },
18615 ],
18616 skills,
18617 statuses: statuses.length > 0 ? statuses : undefined,
18618 };
18619 }
18620 import * as PIXI from 'pixi.js';
18621 import { makeText, textStyle } from '@theme/typography';
18622 import type { SkillSpec } from '@data/skillCatalog';
18623 import { describeReach, describeSkillSpec } from '@data/skillText';
18624 import { getSkillMod, isExclusiveMod, type SkillModRarity } from '@data/skillModCatalog';
18625 import { createUiIcon } from '@view/renderHelpers';
18626 /**
18627  * 单位信息面板：布阵页和战斗页共用同一块渲染。
18628  *
18629  * 两边看的是同一件事--「这个人现在什么状态、带什么招」。做成两份的话，
18630  * 迟早出现布阵页写着个数、战斗里点开写着另一个数，而玩家没法判断该信谁。
18631  * 所以这里只认一个**展示模型**，数值怎么算是调用方（`unitInfoModel.ts`）的事。
18632  */
18633 export interface UnitInfoStat {
18634   label: string;
18635   value: string;
18636 }
18637 export interface UnitInfoSkillSection {
18638   /** 段标题，如「装备技能」「临时技能（本局）」 */
18639   title: string;
18640   name: string;
18641   nameColor: number;
18642   iconKey: string;
18643   /** **折进词条之后**的规格：面板写的必须就是战斗里结算的 */
18644   spec: SkillSpec;
18645   /** 原始规格，只用来标出冷却被词条缩短了 */
18646   baseSpec: SkillSpec;
18647   /** 冷却行后面的补充，如战斗中的「剩 2 回合」 */
18648   cooldownNote?: string;
18649   /** 画范围格子图。临时技能不画，否则面板长到要滚动 */
18650   showRange: boolean;
18651   extraDesc?: string[];
18652   /** 「本局加成」清单挂在这一段的格子图旁边；只给主技能段 */
18653   modIds?: readonly string[];
18654 }
18655 export interface UnitInfoModel {
18656   name: string;
18657   subtitle: string;
```



```
18658  /**
18659  * 48px 头像的**工厂**，不是画好的对象。
18660  *
18661  * 模型要能在没有 DOM 的环境里构造（单测拿它核对面板数值和战斗结算是否一致），
18662  * 而 `createUnitToken` 会 new Graphics，那一步必须推迟到真要渲染的时候。
18663  */
18664  createPortrait: () => PIXI.Container;
18665  stats: UnitInfoStat[];
18666  strikeTitle: string;
18667  strike: UnitInfoStat[];
18668  skills: UnitInfoSkillSection[];
18669  /** 战斗中生效的限时状态，如「中毒: 每回合 -6 血（剩 2 回合）」 */
18670  statuses?: string[];
18671  }
18672  const LABEL_STYLE = textStyle('caption', { fill: 0xaa0088 });
18673  const VALUE_STYLE = textStyle('uiStrong', { fill: 0x3a3a2a, fontSize: 12 });
18674  const SECTION_STYLE = textStyle('uiStrong', { fill: 0x6b4c2a, fontSize: 13 });
18675  const LINE_H = 20;
18676  /** 词条稀有度 -> 米色面板上的字色（三选一卡是深底，那套色直接搬过来会发飘） */
18677  const MOD_TEXT_COLOR: Record<SkillModRarity, number> = {
18678    common: 0x5a6a7a,
18679    rare: 0x2f6fae,
18680    epic: 0xa5561f,
18681  };
18682  /**
18683  * 「本局加成」清单：图标 + 名称×层数 + 这一层的实际效果。
18684  *
18685  * 只写名字不够--「锋锐×2」到底是 +25% 还是 +50%，玩家没法从名字里读出来，
18686  * 而这正是他决定要不要再叠一层的依据，所以每条都带上按层数算好的描述。
18687  *
18688  * 挂不上当前技能的词条（比如给纯治疗技挂「淬毒」）也列，压暗并注明原因：
18689  * 词条是跟着角色走的，换回能用的技能它就活了，直接藏掉会让玩家以为东西丢了。
18690  */
18691  function buildRunModList(
18692    modIds: readonly string[],
18693    spec: SkillSpec,
18694    width: number,
18695  ): PIXI.Container | null {
18696    if (modIds.length === 0) return null;
18697    const counted = new Map<string, number>();
18698    for (const id of modIds) counted.set(id, (counted.get(id) ?? 0) + 1);
18699    const box = new PIXI.Container();
18700    const title = makeText('本局加成', 'caption', { fill: 0x6b4c2a, fontSize: 10, fontWeight: 'bold' });
18701    box.addChild(title);
18702    let y = title.height + 4;
18703    for (const [modId, rawN] of counted) {
18704      const mod = getSkillMod(modId);
18705      if (!mod) continue;
18706      const n = Math.min(rawN, mod.maxStacks);
18707      const live = mod.canApply(spec);
```

```

18708   const row = new PIXI.Container();
18709   row.y = y;
18710   row.alpha = live ? 1 : 0.5;
18711   const icon = createUiIcon(mod.icon, 14);
18712   if (icon) {
18713     icon.y = -1;
18714     row.addChild(icon);
18715   }
18716   const textX = icon ? 18 : 0;
18717   // 专属词条标出来: 它换个技能就再也拿不到, 玩家换招前得知道自己会丢什么
18718   const label = isExclusiveMod(mod) ? `${mod.name} (专属)` : n > 1 ? `${mod.name} × ${n}` : mod.name;
18719   const name = makeText(label, 'caption', {
18720     fill: live ? MOD_TEXT_COLOR[mod.rarity] : 0x8a8a7a,
18721     fontSize: 10,
18722     fontWeight: 'bold',
18723   });
18724   name.x = textX;
18725   row.addChild(name);
18726   const desc = makeText(live ? mod.describe(n) : '当前技能用不到, 换回可用技能即恢复', 'micro', {
18727     fill: 0x7a7a6a,
18728     lineHeight: 12,
18729     wordWrap: true,
18730     wordWrapWidth: Math.max(60, width - textX),
18731   });
18732   desc.x = textX;
18733   desc.y = name.height + 1;
18734   row.addChild(desc);
18735   box.addChild(row);
18736   y += name.height + 1 + desc.height + 5;
18737 }
18738 // 一条都挂不上时只剩个标题, 不如不画
18739 return box.children.length > 1 ? box : (box.destroy({ children: true }), null);
18740 }
18741 type CellKind = 'empty' | 'center' | 'hit' | 'ray';
18742 /** 技能范围示意格 + 右侧的本局加成 + 下方的范围说明与图例 */
18743 function buildRangeRow(
18744   spec: SkillSpec,
18745   modIds: readonly string[] | undefined,
18746   panelW: number,
18747   onTick: (cells: PIXI.Graphics[]) => void,
18748 ): { view: PIXI.Container; height: number } {
18749   const shape = spec.shape;
18750   const cs = 12;
18751   const gap = 1;
18752   const st2 = cs + gap;
18753   let gridR = 2;
18754   let rangeDesc = "";
18755   const isLine = shape.type === 'lineBestRayAllFoes';
18756   if (shape.type === 'neighborAoE') {
18757     gridR = shape.manhattan + 1;

```

```

18758   rangeDesc = `周围${shape.manhattan}格范围\n命中所有敌人`;
18759   } else if (shape.type === 'discAoE') {
18760     // 「横扫」「势不可挡」把环摊成整片圆，格子图必须跟着变--
18761     // 词条改了形状却画老图的话，玩家会照着错的范围走位。
18762     gridR = shape.radius + 1;
18763     rangeDesc = `周围${shape.radius}格全覆盖\n命中所有敌人`;
18764   } else if (shape.type === 'neighborPickLowest') {
18765     gridR = shape.manhattan + 1;
18766     rangeDesc = `${describeReach(shape.manhattan, 'exact')}\n选中血量最低的敌人`;
18767   } else if (shape.type === 'neighborPickFoe') {
18768     gridR = shape.manhattan + 1;
18769     const pickLabel = shape.pick === 'lowestHp' ? '血量最低' : '血量最高';
18770     rangeDesc = `${describeReach(shape.manhattan, shape.reach)}\n选中${pickLabel}的敌人`;
18771   } else if (shape.type === 'neighborPickAlly') {
18772     gridR = shape.manhattan + 1;
18773     const pickLabel = shape.pick === 'lowestHp' ? '血量最低' : '血量最高';
18774     rangeDesc = `周围${shape.manhattan}格范围\n选中${pickLabel}的友方`;
18775   } else if (shape.type === 'selfCast') {
18776     gridR = 1;
18777     rangeDesc = '对自己释放\n无需选择目标';
18778   } else if (isLine) {
18779     gridR = 3;
18780     rangeDesc = '上下左右四方向\n射线穿透所有敌人';
18781   }
18782   const gridD = gridR * 2 + 1;
18783   const cells: CellKind[][] = [];
18784   for (let gy = 0; gy < gridD; gy++) {
18785     cells.push([]);
18786     for (let gx = 0; gx < gridD; gx++) cells[gy].push('empty');
18787   }
18788   cells[gridR][gridR] = 'center';
18789   if (shape.type === 'neighborAoE' || shape.type === 'neighborPickLowest'
18790     || shape.type === 'neighborPickFoe' || shape.type === 'neighborPickAlly'
18791     || shape.type === 'discAoE') {
18792     const md = shape.type === 'discAoE' ? shape.radius : shape.manhattan;
18793     /*
18794     * 整片（曼哈顿 <= md）还是一圈环（正好 = md）。
18795     *
18796     * 这里原先无条件按整片画，而除`discAoE`和`reach: 'within'`以外的形状都是环：
18797     * 「长驱突刺」（正好 2 格）的图因此多画了 4 个贴脸格，玩家照图走到敌人旁边，
18798     * 技能却点不出来。md 为 1 时环和整片恰好一样，所以这个错一直被邻格技能盖着，
18799     * 只有 2 格以上的技能会露出来。
18800     */
18801     const solid = shape.type === 'discAoE'
18802       || (shape.type === 'neighborPickFoe' && shape.reach === 'within');
18803     for (let dy = -md; dy <= md; dy++) {
18804       for (let dx = -md; dx <= md; dx++) {
18805         if (dx === 0 && dy === 0) continue;
18806         const d = Math.abs(dx) + Math.abs(dy);
18807         if (solid ? d <= md : d === md) cells[gridR + dy][gridR + dx] = 'hit';

```

```

18808     }
18809     }
18810   } else if (isLine) {
18811     for (const [ddx, ddy] of [[0, -1], [0, 1], [-1, 0], [1, 0]]) {
18812       for (let s = 1; s <= gridR; s++) {
18813         const gx = gridR + ddx! * s;
18814         const gy = gridR + ddy! * s;
18815         if (gx >= 0 && gx < gridD && gy >= 0 && gy < gridD) cells[gy][gx] = 'ray';
18816       }
18817     }
18818   }
18819   const gridTotalW = gridD * st2 - gap;
18820   const row = new PIXI.Container();
18821   const gridContainer = new PIXI.Container();
18822   const hitCells: PIXI.Graphics[] = [];
18823   for (let gy = 0; gy < gridD; gy++) {
18824     for (let gx = 0; gx < gridD; gx++) {
18825       const kind = cells[gy][gx];
18826       const px = gx * st2;
18827       const py = gy * st2;
18828       const cell = new PIXI.Graphics();
18829       if (kind === 'center') {
18830         cell.beginFill(0x4488cc, 0.85);
18831         cell.drawRoundedRect(px, py, cs, cs, 2);
18832         cell.endFill();
18833       } else if (kind === 'hit' || kind === 'ray') {
18834         cell.beginFill(kind === 'ray' ? 0xdd6633 : 0xcc3333, 0.7);
18835         cell.drawRoundedRect(px, py, cs, cs, 2);
18836         cell.endFill();
18837         hitCells.push(cell);
18838       } else {
18839         cell.lineStyle(1, 0xccccbb, 0.25);
18840         cell.beginFill(0xeeeedd, 0.12);
18841         cell.drawRoundedRect(px, py, cs, cs, 1);
18842         cell.endFill();
18843       }
18844       gridContainer.addChild(cell);
18845     }
18846   }
18847   // 射线技能在四个边缘画箭头，表示延伸
18848   if (isLine) {
18849     const arrowStyle = textStyle('micro', { fill: 0xdd6633, fontSize: 8 });
18850     const arrowOffsets: [number, number, string][] = [
18851       [gridR * st2 + cs / 2, -6, '▲'],
18852       [gridR * st2 + cs / 2, gridD * st2 - gap + 1, '▼'],
18853       [-6, gridR * st2 + cs / 2, '<'],
18854       [gridD * st2 - gap + 2, gridR * st2 + cs / 2, '>'],
18855     ];
18856     for (const [ax, ay, ch] of arrowOffsets) {
18857       const ar = new PIXI.Text(ch, arrowStyle);

```

```
18858     ar.anchor.set(0.5);
18859     ar.x = ax;
18860     ar.y = ay;
18861     gridContainer.addChild(ar);
18862 }
18863 }
18864 gridContainer.x = 16;
18865 row.addChild(gridContainer);
18866 const colX = 16 + gridTotalW + 12;
18867 const colW = panelW - 12 - colX;
18868 // 本局词条：占掉格子图右边那块空地。
18869 //
18870 // 之前只有背包里能翻到，而玩家真正想核对「这一局攒了什么」的时机，恰恰是开打前
18871 // 对着这个面板看阵容。摆在技能格子旁边还顺带说清了归属：改的就是上面那一招。
18872 const modsBlock = modIds ? buildRunModList(modIds, spec, colW) : null;
18873 // 词条把右列占了的话，范围说明和图例挪到格子下面排成一行。
18874 // 反过来把词条压到下面是不行的：右列只剩说明文字下面那几像素，
18875 // 一条「技能伤害提升 50%」就要折三行。
18876 const descBelow = modsBlock !== null;
18877 const rangeDescTx = makeText(descBelow ? rangeDesc.replace(/\n/g, ' · '): rangeDesc, 'caption', {
18878     fill: 0x6a6a5a, fontSize: 10, lineHeight: 15,
18879     wordWrap: true, wordWrapWidth: descBelow ? panelW - 32 : colW,
18880 });
18881 if (descBelow) {
18882     rangeDescTx.x = 16;
18883     // 让到词条那一列的下面，而不是只让过格子图：词条攒多了会比格子高，
18884     // 按格子高度排会直接压在最后一条词条上。
18885     rangeDescTx.y = Math.max(gridTotalW, modsBlock?.height ?? 0) + 8;
18886 } else {
18887     rangeDescTx.x = colX;
18888     rangeDescTx.y = Math.max(0, (gridTotalW - rangeDescTx.height) / 2);
18889 }
18890 row.addChild(rangeDescTx);
18891 if (modsBlock) {
18892     modsBlock.x = colX;
18893     row.addChild(modsBlock);
18894 }
18895 // 图例。跟着说明走：在右列时排在它下面，在下方时右对齐到同一行
18896 const legendY = descBelow
18897     ? rangeDescTx.y + Math.max(0, rangeDescTx.height - 12)
18898     : rangeDescTx.y + rangeDescTx.height + 6;
18899 const legendX = descBelow
18900     ? Math.max(rangeDescTx.x + rangeDescTx.width + 12, panelW - 12 - 78)
18901     : colX;
18902 const legCenterDot = new PIXI.Graphics();
18903 legCenterDot.beginFill(0x4488cc, 0.85);
18904 legCenterDot.drawRoundedRect(0, 0, 8, 8, 2);
18905 legCenterDot.endFill();
18906 legCenterDot.x = legendX;
18907 legCenterDot.y = legendY;
```

```
18908 row.addChild(legCenterDot);
18909 const legCenterTx = makeText('自身', 'micro', { fill: 0x888877 });
18910 legCenterTx.x = legendX + 12;
18911 legCenterTx.y = legendY - 1;
18912 row.addChild(legCenterTx);
18913 const legHitDot = new PIXI.Graphics();
18914 legHitDot.beginFill(isLine ? 0xdd6633 : 0xcc3333, 0.7);
18915 legHitDot.drawRoundedRect(0, 0, 8, 8, 2);
18916 legHitDot.endFill();
18917 legHitDot.x = legCenterTx.x + legCenterTx.width + 10;
18918 legHitDot.y = legendY;
18919 row.addChild(legHitDot);
18920 const legHitTx = makeText('范围', 'micro', { fill: 0x888877 });
18921 legHitTx.x = legHitDot.x + 12;
18922 legHitTx.y = legendY - 1;
18923 row.addChild(legHitTx);
18924 onTick(hitCells);
18925 const height = Math.max(
18926     gridTotalW,
18927     legendY + 14,
18928     rangeDescTx.y + rangeDescTx.height,
18929     modsBlock ? modsBlock.height : 0,
18930 );
18931 return { view: row, height };
18932 }
18933 function separator(panelW: number, y: number): PIXI.Graphics {
18934     const g = new PIXI.Graphics();
18935     g.lineStyle(1, 0xd0c8b8, 0.6);
18936     g.moveTo(12, y);
18937     g.lineTo(panelW - 12, y);
18938     return g;
18939 }
18940 /** 两列键值区, 返回占用高度 */
18941 function addStatGrid(
18942     panel: PIXI.Container,
18943     items: UnitInfoStat[],
18944     panelW: number,
18945     top: number,
18946 ): number {
18947     const colW = Math.floor((panelW - 24) / 2);
18948     for (let i = 0; i < items.length; i++) {
18949         const s = items[i];
18950         const sx = 16 + (i % 2) * colW;
18951         const sy = top + Math.floor(i / 2) * LINE_H;
18952         const lb = new PIXI.Text(s.label, LABEL_STYLE);
18953         lb.x = sx;
18954         lb.y = sy;
18955         panel.addChild(lb);
18956         const vl = new PIXI.Text(s.value, VALUE_STYLE);
18957         vl.x = sx + 36;
```

```
18958     vl.y = sy;
18959     panel.addChild(vl);
18960 }
18961 return Math.ceil(items.length / 2) * LINE_H;
18962 }
18963 export interface UnitInfoPanel {
18964     view: PIXI.Container;
18965     height: number;
18966     /** 范围格子的呼吸动画；面板销毁时要调 */
18967     stop(): void;
18968 }
18969 export interface UnitInfoPanelOptions {
18970     /**
18971      * 画自己的米白底（默认画）。
18972      *
18973      * 嵌进另一个米白面板时关掉：两层同色圆角矩形叠在一起只会在拐角处露出一圈
18974      * 深浅不一的边，读起来像没对齐。角色页的详情弹窗就是这种嵌套。
18975      */
18976     drawBg?: boolean;
18977 }
18978 export function createUnitInfoPanel(
18979     model: UnitInfoModel,
18980     panelW: number,
18981     opts?: UnitInfoPanelOptions,
18982 ): UnitInfoPanel {
18983     const panel = new PIXI.Container();
18984     panel.eventMode = 'static';
18985     const tickers: (() => void)[] = [];
18986     let cy = 16;
18987     const portrait = model.createPortrait();
18988     portrait.x = 30;
18989     portrait.y = cy + 24;
18990     panel.addChild(portrait);
18991     const nameTx = makeText(model.name, 'title', { fill: 0x3a3a2a, fontSize: 16 });
18992     nameTx.x = 62;
18993     nameTx.y = cy + 6;
18994     panel.addChild(nameTx);
18995     const subTx = makeText(model.subtitle, 'body', { fill: 0x8a7a5a });
18996     subTx.x = 62;
18997     subTx.y = cy + 28;
18998     panel.addChild(subTx);
18999     cy += 56;
19000     panel.addChild(separator(panelW, cy));
19001     cy += 8;
19002     const secBase = new PIXI.Text('基础属性', SECTION_STYLE);
19003     secBase.x = 12;
19004     secBase.y = cy;
19005     panel.addChild(secBase);
19006     cy += LINE_H + 2;
19007     cy += addStatGrid(panel, model.stats, panelW, cy) + 8;
```

```
19008 // 限时状态。战斗中点开才有内容--「他为什么突然打这么疼」只能在这里回答。
19009 if (model.statuses?.length) {
19010     panel.addChild(separator(panelW, cy));
19011     cy += 8;
19012     const secSt = new PIXI.Text('当前状态', SECTION_STYLE);
19013     secSt.x = 12;
19014     secSt.y = cy;
19015     panel.addChild(secSt);
19016     cy += LINE_H + 2;
19017     const tx = makeText(model.statuses.join('\n'), 'caption', {
19018         fill: 0x555544, fontSize: 10, lineHeight: 16,
19019         wordWrap: true, wordWrapWidth: panelW - 32,
19020     });
19021     tx.x = 16;
19022     tx.y = cy;
19023     panel.addChild(tx);
19024     cy += tx.height + 8;
19025 }
19026 panel.addChild(separator(panelW, cy));
19027 cy += 8;
19028 const secStrike = new PIXI.Text(model.strikeTitle, SECTION_STYLE);
19029 secStrike.x = 12;
19030 secStrike.y = cy;
19031 panel.addChild(secStrike);
19032 cy += LINE_H + 2;
19033 cy += addStatGrid(panel, model.strike, panelW, cy) + 8;
19034 for (const sk of model.skills) {
19035     panel.addChild(separator(panelW, cy));
19036     cy += 8;
19037     const sec = new PIXI.Text(sk.title, SECTION_STYLE);
19038     sec.x = 12;
19039     sec.y = cy;
19040     panel.addChild(sec);
19041     cy += LINE_H + 2;
19042     // 技能图标：战斗操作条、三选一卡片、这里用的是同一张图。
19043     // 玩家在这三处认的是同一个符号，换到战斗里那排无字圆钮才不用重新学。
19044     const icon = createUiIcon(sk.iconKey, 22);
19045     if (icon) {
19046         icon.x = 16;
19047         icon.y = cy - 3;
19048         panel.addChild(icon);
19049     }
19050     const nameX = icon ? 42 : 16;
19051     const skName = makeText(sk.name, 'uiStrong', { fill: sk.nameColor, fontSize: 13 });
19052     skName.x = nameX;
19053     skName.y = cy;
19054     panel.addChild(skName);
19055     const cdText = `CD: ${sk.spec.cooldown}回合${sk.cooldownNote ?? ''}`;
19056     const cdTx = makeText(cdText, 'caption', {
19057         fill: sk.spec.cooldown < sk.baseSpec.cooldown ? 0x3a8a5a : 0x888888,
```



```
19058 });
19059 cdTx.x = nameX + skName.width + 10;
19060 cdTx.y = cy + 2;
19061 panel.addChild(cdTx);
19062 cy += LINE_H;
19063 const descTx = makeText([...describeSkillSpec(sk.spec), ...(sk.extraDesc ?? [])].join('\n'), ...
19064   fill: 0x555544, fontSize: 10, lineHeight: 16,
19065   wordWrap: true, wordWrapWidth: panelW - 32,
19066 );
19067 descTx.x = 16;
19068 descTx.y = cy;
19069 panel.addChild(descTx);
19070 cy += descTx.height + 8;
19071 if (sk.showRange) {
19072   const { view, height } = buildRangeRow(sk.spec, sk.modIds, panelW, (hitCells) => {
19073     let phase = 0;
19074     tickers.push() => {
19075       phase += 0.06;
19076       const a = 0.45 + 0.35 * Math.sin(phase);
19077       for (const c of hitCells) c.alpha = a;
19078     });
19079   });
19080   view.y = cy;
19081   panel.addChild(view);
19082   cy += height + 8;
19083 }
19084 }
19085 cy += 12;
19086 if (opts?.drawBg ?? true) {
19087   const bg = new PIXI.Graphics();
19088   bg.beginFill(0xfefef6, 0.97);
19089   bg.drawRoundedRect(0, 0, panelW, cy, 14);
19090   bg.endFill();
19091   panel.addChildAt(bg, 0);
19092 }
19093 // 场景可能被 SceneManager 整棵拆掉而不经 `stop()` (战斗结束时面板还开着),
19094 // 所以 ticker 自己也要能发现容器没了并摘掉自己, 否则它会一直抓着这棵树不放。
19095 const tick = (): void => {
19096   if (panel.destroyed) {
19097     PIXI.Ticker.shared.remove(tick);
19098     return;
19099   }
19100   for (const t of tickers) t();
19101 };
19102 if (tickers.length > 0) PIXI.Ticker.shared.add(tick);
19103 return {
19104   view: panel,
19105   height: cy,
19106   stop: () => PIXI.Ticker.shared.remove(tick),
19107 };
```

```
19108 }
19109 /**
19110 * 遮罩 + 居中面板 + 点遮罩关闭。两个页面都要这一套，别各写一遍。
19111 *
19112 * 面板自己吃掉点击（`eventMode: 'static'`），否则点面板正文也会把它关掉。
19113 */
19114 export function createUnitInfoOverlay(
19115   model: UnitInfoModel,
19116   screenW: number,
19117   screenH: number,
19118   onClose: () => void,
19119 ): { view: PIXI.Container; stop(): void } {
19120   const root = new PIXI.Container();
19121   const dim = new PIXI.Graphics();
19122   dim.beginFill(0x000000, 0.5);
19123   dim.drawRect(0, 0, screenW, screenH);
19124   dim.endFill();
19125   dim.eventMode = 'static';
19126   dim.on('pointertap', onClose);
19127   root.addChild(dim);
19128   const panelW = Math.min(300, screenW - 32);
19129   const panel = createUnitInfoPanel(model, panelW);
19130   panel.view.x = Math.floor((screenW - panelW) / 2);
19131   // 面板可能比屏幕高（词条攒满 + 长技能说明），那就贴顶而不是居中溢出到看不见
19132   panel.view.y = Math.max(8, Math.floor((screenH - panel.height) / 2));
19133   root.addChild(panel.view);
19134   return { view: root, stop: panel.stop };
19135 }
19136 import * as PIXI from 'pixi.js';
19137 import { makeText } from '@theme/typography';
19138 export interface UnitOverheadOptions {
19139   maxHp: number;
19140   currentHp: number;
19141   professionName: string;
19142   faction: 'player' | 'enemy';
19143   cell: number;
19144 }
19145 export interface UnitOverheadHandle {
19146   readonly root: PIXI.Container;
19147   updateHp: (currentHp: number, maxHp?: number) => void;
19148   destroy: () => void;
19149 }
19150 const HP_BG = 0x1a0505;
19151 const PLAYER_HP_BORDER = 0x2d8a2d;
19152 const PLAYER_HP_FILL = 0x44bb44;
19153 const ENEMY_HP_BORDER = 0xcc3333;
19154 const ENEMY_HP_FILL = 0xb82a2a;
19155 /**
19156 * 单位头顶：血条 + 职业名（同一行，职业在血条右侧）。
19157 * 整体在角色上方，本地坐标 y=0 为底边。
```

```
19158 */
19159 export function createUnitOverhead(opts: UnitOverheadOptions): UnitOverheadHandle {
19160   const root = new PIXI.Container();
19161   let maxHp = Math.max(1, opts.maxHp);
19162   let curHp = Math.min(Math.max(0, opts.currentHp), maxHp);
19163   const isPlayer = opts.faction === 'player';
19164   const hpBorder = isPlayer ? PLAYER_HP_BORDER : ENEMY_HP_BORDER;
19165   const hpFillColor = isPlayer ? PLAYER_HP_FILL : ENEMY_HP_FILL;
19166   const barW = Math.max(18, Math.floor(opts.cell * 0.52));
19167   const barH = Math.max(3, Math.floor(opts.cell * 0.09));
19168   const labelFs = Math.max(7, Math.min(10, Math.floor(opts.cell * 0.18)));
19169   const label = makeText(opts.professionName, 'combatLabel', {
19170     fill: 0xffffffff,
19171     fontSize: labelFs,
19172   });
19173   label.anchor.set(0, 0.5);
19174   const totalW = barW + 3 + label.width;
19175   const hpBg = new PIXI.Graphics();
19176   const hpFill = new PIXI.Graphics();
19177   function redrawBar(): void {
19178     hpBg.clear();
19179     hpFill.clear();
19180     const x0 = -totalW / 2;
19181     const y0 = -barH;
19182     hpBg.lineStyle(1, hpBorder, 1);
19183     hpBg.beginFill(HP_BG, 0.95);
19184     hpBg.drawRoundedRect(x0, y0, barW, barH, 2);
19185     hpBg.endFill();
19186     const ratio = maxHp > 0 ? Math.max(0, Math.min(1, curHp / maxHp)) : 0;
19187     const innerW = barW - 4;
19188     const fillW = Math.max(0, innerW * ratio);
19189     hpFill.beginFill(hpFillColor, 1);
19190     hpFill.drawRoundedRect(x0 + 2, y0 + 1, fillW, barH - 2, 1);
19191     hpFill.endFill();
19192   }
19193   label.x = -totalW / 2 + barW + 3;
19194   label.y = -barH / 2;
19195   root.addChild(hpBg);
19196   root.addChild(hpFill);
19197   root.addChild(label);
19198   redrawBar();
19199   return {
19200     root,
19201     updateHp(currentHp: number, nextMax?: number): void {
19202       if (nextMax !== undefined) maxHp = Math.max(1, nextMax);
19203       curHp = Math.min(Math.max(0, currentHp), maxHp);
19204       redrawBar();
19205     },
19206     destroy(): void {
19207       root.destroy({ children: true });
```

```
19208     },
19209   };
19210 }
19211 /**
19212  * Image 构造函数模拟
19213  */
19214 const platform = require('./platform');
19215 function Image() {
19216   return platform.createImage();
19217 }
19218 module.exports = Image;
19219 /**
19220  * 触摸事件适配
19221  * 将小游戏触摸事件转换为 PixiJS 所需的标准 DOM 事件格式
19222  */
19223 const platform = require('./platform');
19224 const { canvas } = require('./canvas');
19225 class TouchEvent {
19226   constructor(type, touches) {
19227     this.type = type;
19228     this.target = canvas;
19229     this.currentTarget = canvas;
19230     this.touches = touches || [];
19231     this.changedTouches = touches || [];
19232     this.targetTouches = touches || [];
19233     this.timeStamp = Date.now();
19234     this.bubbles = true;
19235     this.cancelable = true;
19236     this.defaultPrevented = false;
19237   }
19238   preventDefault() {
19239     this.defaultPrevented = true;
19240   }
19241   stopPropagation() {}
19242 }
19243 // 将小游戏触摸坐标转换为 canvas 坐标
19244 function convertTouches(rawTouches) {
19245   if (!rawTouches) return [];
19246   return Array.prototype.map.call(rawTouches, (touch) => ({
19247     identifier: touch.identifier,
19248     clientX: touch.clientX,
19249     clientY: touch.clientY,
19250     pageX: touch.clientX,
19251     pageY: touch.clientY,
19252     screenX: touch.clientX,
19253     screenY: touch.clientY,
19254     target: canvas,
19255   }));
19256 }
19257 // 注册触摸事件监听桥接
```

```
19258 function registerTouchEvents() {
19259   const _listeners = {};
19260   // 给 canvas 挂上 addEventListener / removeEventListener
19261   canvas.addEventListener = function(type, handler, options) {
19262     if (!_listeners[type]) _listeners[type] = [];
19263     _listeners[type].push(handler);
19264   };
19265   canvas.removeEventListener = function(type, handler) {
19266     if (!_listeners[type]) return;
19267     const idx = _listeners[type].indexOf(handler);
19268     if (idx !== -1) _listeners[type].splice(idx, 1);
19269   };
19270   // 分发事件到 canvas 上的监听器
19271   function dispatch(type, rawEvent) {
19272     const touches = convertTouches(rawEvent.touches || rawEvent.changedTouches);
19273     const event = new TouchEvent(type, touches);
19274     event.changedTouches = convertTouches(rawEvent.changedTouches);
19275     const queue = _listeners[type];
19276     if (queue) {
19277       queue.forEach(handler => {
19278         try { handler(event); } catch (e) { console.error('[TouchEvent]', type, e); }
19279       });
19280     }
19281   }
19282   // pointer 事件映射 (PixiJS 7 优先使用 pointer 事件)
19283   function dispatchPointer(pointerType, rawEvent) {
19284     const touches = rawEvent.changedTouches || rawEvent.touches || [];
19285     if (!touches.length) return;
19286     const touch = touches[0];
19287     const pointerEvent = {
19288       type: pointerType,
19289       pointerId: touch.identifier || 0,
19290       pointerType: 'touch',
19291       clientX: touch.clientX,
19292       clientY: touch.clientY,
19293       pageX: touch.clientX,
19294       pageY: touch.clientY,
19295       screenX: touch.clientX,
19296       screenY: touch.clientY,
19297       x: touch.clientX,
19298       y: touch.clientY,
19299       offsetX: touch.clientX,
19300       offsetY: touch.clientY,
19301       width: 1,
19302       height: 1,
19303       pressure: pointerType === 'pointerup' ? 0 : 0.5,
19304       button: 0,
19305       buttons: pointerType === 'pointerup' ? 0 : 1,
19306       isPrimary: true,
19307       target: canvas,
```

```

19308     currentTarget: canvas,
19309     timeStamp: Date.now(),
19310     bubbles: true,
19311     cancelable: true,
19312     preventDefault() {},
19313     stopPropagation() {},
19314     stopImmediatePropagation() {},
19315 };
19316 const queue = _listeners[pointerType];
19317 if (queue) {
19318     queue.forEach(handler => {
19319         try { handler(pointerEvent); } catch (e) { console.error('[PointerEvent]', pointerType, e...
19320     });
19321 }
19322 }
19323 // 分发 window 上的全局事件 (PixiJS EventSystem 在 window 上也注册了 pointermove/pointerup)
19324 function dispatchToWindow(type, event) {
19325     if (typeof GameGlobal !== 'undefined' && GameGlobal.__windowDispatchEvent) {
19326         GameGlobal.__windowDispatchEvent(type, event);
19327     }
19328 }
19329 // 诊断: 3秒后输出已注册的 canvas 事件监听器类型
19330 setTimeout(function() {
19331     console.log('[TouchEvent] canvas listeners:', Object.keys(_listeners).join(', ') || '(空)');
19332 }, 3000);
19333 var _touchLogCount = 0;
19334 platform.onTouchStart((e) => {
19335     _touchLogCount++;
19336     if (_touchLogCount <= 5) {
19337         var t = (e.changedTouches || e.touches || []).length;
19338         console.log('[Touch] down #' + _touchLogCount,
19339             'x:', t && t.clientX, 'y:', t && t.clientY,
19340             'canvasListeners:', Object.keys(_listeners).join(','));
19341     }
19342     dispatch('touchstart', e);
19343     dispatchPointer('pointerdown', e);
19344     // window 上也需要收到 pointerdown
19345     var touches = e.changedTouches || e.touches || [];
19346     if (touches.length) {
19347         var t = touches[0];
19348         dispatchToWindow('pointerdown', {
19349             type: 'pointerdown', pointerId: t.identifier || 0, pointerType: 'touch',
19350             clientX: t.clientX, clientY: t.clientY, pageX: t.clientX, pageY: t.clientY,
19351             button: 0, buttons: 1, isPrimary: true, target: canvas,
19352             preventDefault: function() {}, stopPropagation: function() {}, stopImmediatePropagation: fu...
19353         });
19354     }
19355 });
19356 var _moveLogCount = 0;
19357 platform.onTouchMove((e) => {

```

```
19358 dispatch('touchmove', e);
19359 dispatchPointer('pointermove', e);
19360 var touches = e.changedTouches || e.touches || [];
19361 if (touches.length) {
19362     var t = touches[0];
19363     _moveLogCount++;
19364     if (_moveLogCount <= 3) {
19365         console.log('[Touch] move #' + _moveLogCount,
19366             'x:', t.clientX, 'y:', t.clientY,
19367             'windowListeners(pointermove):', typeof GameGlobal.__windowDispatchEvent);
19368     }
19369     dispatchToWindow('pointermove', {
19370         type: 'pointermove', pointerId: t.identifier || 0, pointerType: 'touch',
19371         clientX: t.clientX, clientY: t.clientY, pageX: t.clientX, pageY: t.clientY,
19372         button: 0, buttons: 1, isPrimary: true, target: canvas,
19373         preventDefault: function() {}, stopPropagation: function() {}, stopImmediatePropagation: fu...
19374     });
19375 }
19376 });
19377 platform.onTouchEnd((e) => {
19378     dispatch('touchend', e);
19379     dispatchPointer('pointerup', e);
19380     var touches = e.changedTouches || [];
19381     if (touches.length) {
19382         var t = touches[0];
19383         dispatchToWindow('pointerup', {
19384             type: 'pointerup', pointerId: t.identifier || 0, pointerType: 'touch',
19385             clientX: t.clientX, clientY: t.clientY, pageX: t.clientX, pageY: t.clientY,
19386             button: 0, buttons: 0, isPrimary: true, target: canvas,
19387             preventDefault: function() {}, stopPropagation: function() {}, stopImmediatePropagation: fu...
19388         });
19389     }
19390 });
19391 platform.onTouchCancel((e) => {
19392     dispatch('touchcancel', e);
19393     dispatchPointer('pointercancel', e);
19394     var touches = e.changedTouches || [];
19395     if (touches.length) {
19396         var t = touches[0];
19397         dispatchToWindow('pointercancel', {
19398             type: 'pointercancel', pointerId: t.identifier || 0, pointerType: 'touch',
19399             clientX: t.clientX, clientY: t.clientY, pageX: t.clientX, pageY: t.clientY,
19400             button: 0, buttons: 0, isPrimary: true, target: canvas,
19401             preventDefault: function() {}, stopPropagation: function() {}, stopImmediatePropagation: fu...
19402         });
19403     }
19404 });
19405 // canvas.getBoundingClientRect - PixiJS 用来计算事件坐标
19406 // 必须返回逻辑像素尺寸（与触摸事件 clientX/clientY 一致），否则坐标偏移
19407 var sysInfo = platform.getSystemInfoSync();
```

```
19408 var screenW = sysInfo.screenWidth || sysInfo.windowWidth || 375;
19409 var screenH = sysInfo.screenHeight || sysInfo.windowHeight || 667;
19410 try {
19411   canvas.getBoundingClientRect = function() {
19412     return {
19413       x: 0,
19414       y: 0,
19415       top: 0,
19416       left: 0,
19417       width: screenW,
19418       height: screenH,
19419       right: screenW,
19420       bottom: screenH,
19421     };
19422   };
19423 } catch (e) {}
19424 // clientWidth/clientHeight 也需返回逻辑像素
19425 try {
19426   Object.defineProperty(canvas, 'clientWidth', { get: function() { return screenW; }, configura...
19427   Object.defineProperty(canvas, 'clientHeight', { get: function() { return screenH; }, configur...
19428 } catch (e) {}
19429 // Pixijs 会检查 canvas.style (微信 canvas 部分属性可能只读)
19430 try {
19431   if (!canvas.style) canvas.style = {};
19432   canvas.style.touchAction = "";
19433   canvas.style.msTouchAction = "";
19434   canvas.style.cursor = "";
19435   canvas.style.width = screenW + 'px';
19436   canvas.style.height = screenH + 'px';
19437 } catch (e) {}
19438 // Pixijs 检查 focus
19439 if (!canvas.focus) canvas.focus = function() {};
19440 // Pixijs EventSystem 需要 canvas.parentElement 来 addEventListener
19441 // 不能设为 null, 设为一个带事件能力的虚拟节点
19442 var _parentListeners = {};
19443 var fakeParent = {
19444   addEventListener: function(type, handler, options) {
19445     if (!_parentListeners[type]) _parentListeners[type] = [];
19446     _parentListeners[type].push(handler);
19447   },
19448   removeEventListener: function(type, handler) {
19449     if (!_parentListeners[type]) return;
19450     var idx = _parentListeners[type].indexOf(handler);
19451     if (idx !== -1) _parentListeners[type].splice(idx, 1);
19452   },
19453 };
19454 try { canvas.parentElement = fakeParent; } catch (e) {
19455   try { Object.defineProperty(canvas, 'parentElement', { value: fakeParent, configurable: true,...
19456   }
19457   try { canvas.parentNode = fakeParent; } catch (e) {
```



```
19458   try { Object.defineProperty(canvas, 'parentNode', { value: fakeParent, configurable: true, wr...
19459   }
19460   // 诊断: 确认 parentElement 是否设置成功
19461   console.log('[TouchEvent] canvas.parentElement set:', !!canvas.parentElement,
19462     ', getBoundingClientRect:', typeof canvas.getBoundingClientRect);
19463   }
19464   module.exports = { TouchEvent, registerTouchEvents };
19465   /**
19466    * XMLHttpRequest 模拟
19467    * PixiJS 资源加载可能会用到
19468    */
19469   const platform = require('./platform');
19470   class XMLHttpRequest {
19471     constructor() {
19472       this.readyState = 0;
19473       this.status = 0;
19474       this.statusText = "";
19475       this.responseText = "";
19476       this.response = null;
19477       this.responseType = "";
19478       this.responseURL = "";
19479       this.withCredentials = false;
19480       this.timeout = 0;
19481       this._method = "";
19482       this._url = "";
19483       this._headers = {};
19484       this.onreadystatechange = null;
19485       this.onload = null;
19486       this.onerror = null;
19487       this.onabort = null;
19488       this.ontimeout = null;
19489       this.onprogress = null;
19490     }
19491     open(method, url) {
19492       this._method = method;
19493       this._url = url;
19494       this.readyState = 1;
19495     }
19496     setRequestHeader(key, value) {
19497       this._headers[key] = value;
19498     }
19499     getResponseHeader(key) {
19500       return this._responseHeaders ? this._responseHeaders[key.toLowerCase()] : null;
19501     }
19502     getAllResponseHeaders() {
19503       return "";
19504     }
19505     send(data) {
19506       const self = this;
19507       const responseType = this.responseType || 'text';
```

```
19508 platform.request({
19509   url: this._url,
19510   method: this._method || 'GET',
19511   header: this._headers,
19512   data: data || undefined,
19513   responseType: responseType === 'arraybuffer' ? 'arraybuffer' : 'text',
19514   dataType: responseType === 'json' ? 'json' : undefined,
19515   success(res) {
19516     self.status = res.statusCode;
19517     self.statusText = res.statusCode + " ";
19518     self._responseHeaders = res.header || {};
19519     if (responseType === 'arraybuffer') {
19520       self.response = res.data;
19521     } else if (responseType === 'json') {
19522       self.response = res.data;
19523       self.responseText = JSON.stringify(res.data);
19524     } else {
19525       self.responseText = res.data;
19526       self.response = res.data;
19527     }
19528     self.readyState = 4;
19529     if (self.onreadystatechange) self.onreadystatechange();
19530     if (self.onload) self.onload();
19531   },
19532   fail(err) {
19533     self.status = 0;
19534     self.readyState = 4;
19535     if (self.onreadystatechange) self.onreadystatechange();
19536     if (self.onerror) self.onerror(err);
19537   },
19538 });
19539 }
19540 abort() {
19541   if (this.onabort) this.onabort();
19542 }
19543 addEventListener(type, handler) {
19544   this['on' + type] = handler;
19545 }
19546 removeEventListener() {}
19547 }
19548 // 静态常量
19549 XMLHttpRequest.UNSENT = 0;
19550 XMLHttpRequest.OPENED = 1;
19551 XMLHttpRequest.HEADERS_RECEIVED = 2;
19552 XMLHttpRequest.LOADING = 3;
19553 XMLHttpRequest.DONE = 4;
19554 module.exports = XMLHttpRequest;
19555 /**
19556  * Canvas 管理模块
19557  * 第一次 createCanvas() 返回主屏 canvas（小游戏环境特性，微信和抖音一致）
```

```
19558 */
19559 const platform = require('./platform');
19560 const canvas = platform.createCanvas();
19561 module.exports = { canvas };
19562 /**
19563  * document 对象模拟
19564  */
19565 const platform = require('./platform');
19566 const Image = require('./Image');
19567 const { Element } = require('./element');
19568 const _eventListeners = {};
19569 const body = new Element();
19570 body.clientWidth = 0;
19571 body.clientHeight = 0;
19572 const documentElement = new Element();
19573 const document = {
19574   body,
19575   documentElement,
19576   readyState: 'complete',
19577   createElement(tag) {
19578     tag = tag.toLowerCase();
19579     switch (tag) {
19580       case 'canvas':
19581         return platform.createCanvas();
19582       case 'img':
19583       case 'image':
19584         return platform.createImage();
19585       default:
19586         return new Element();
19587     }
19588   },
19589   createElementNS(_ns, tag) {
19590     return this.createElement(tag);
19591   },
19592   createTextNode() {
19593     return new Element();
19594   },
19595   getElementById() {
19596     return null;
19597   },
19598   getElementsByTagName(tag) {
19599     if (tag === 'canvas') {
19600       const { canvas } = require('./canvas');
19601       return [canvas];
19602     }
19603     return [];
19604   },
19605   querySelector() {
19606     return null;
19607   },
```

```
19608   querySelectorAll() {
19609     return [];
19610   },
19611   // PixiJS EventSystem 使用 elementFromPoint 来确定事件目标
19612   elementFromPoint() {
19613     const { canvas } = require('./canvas');
19614     return canvas;
19615   },
19616   addEventListener(type, handler) {
19617     if (!_eventListeners[type]) _eventListeners[type] = [];
19618     _eventListeners[type].push(handler);
19619   },
19620   removeEventListener(type, handler) {
19621     if (!_eventListeners[type]) return;
19622     const idx = _eventListeners[type].indexOf(handler);
19623     if (idx !== -1) _eventListeners[type].splice(idx, 1);
19624   },
19625   dispatchEvent(ev) {
19626     const queue = _eventListeners[ev.type];
19627     if (queue) queue.forEach(handler => handler(ev));
19628   },
19629   // PixiJS 会检查这些
19630   fonts: {
19631     add() {},
19632     delete() {},
19633     has() { return false; },
19634     forEach() {},
19635   },
19636   hidden: false,
19637   visibilityState: 'visible',
19638 };
19639 module.exports = document;
19640 /**
19641  * DOM 元素模拟
19642  * 业界已知坑: HTMLCanvasElement / HTMLImageElement 必须用 constructor 直接赋值
19643  * 不能用 class extends, 否则 instanceof 校验会失败
19644  */
19645 const platform = require('./platform');
19646 class Element {
19647   constructor() {
19648     this.childNodes = [];
19649     this.style = { cursor: null };
19650     this.clientWidth = 0;
19651     this.clientHeight = 0;
19652     this._listeners = {};
19653   }
19654   appendChild(child) {
19655     this.childNodes.push(child);
19656     return child;
19657   }
```

```
19658 removeChild(child) {
19659   const idx = this.childNodes.indexOf(child);
19660   if (idx !== -1) this.childNodes.splice(idx, 1);
19661   return child;
19662 }
19663 addEventListener(type, handler) {
19664   if (!this._listeners[type]) this._listeners[type] = [];
19665   this._listeners[type].push(handler);
19666 }
19667 removeEventListener(type, handler) {
19668   if (!this._listeners[type]) return;
19669   const idx = this._listeners[type].indexOf(handler);
19670   if (idx !== -1) this._listeners[type].splice(idx, 1);
19671 }
19672 insertBefore() {}
19673 replaceChild() {}
19674 cloneNode() { return new Element(); }
19675 setAttribute() {}
19676 getAttribute() { return null; }
19677 getBoundingClientRect() {
19678   return { x: 0, y: 0, top: 0, left: 0, width: 0, height: 0, right: 0, bottom: 0 };
19679 }
19680 }
19681 // 通过 constructor 直接赋值（非 extends），确保 instanceof 正确
19682 const HTMLCanvasElement = platform.createCanvas().constructor;
19683 const HTMLImageElement = platform.createImage().constructor;
19684 class HTMLVideoElement extends Element {}
19685 module.exports = {
19686   Element,
19687   HTMLCanvasElement,
19688   HTMLImageElement,
19689   HTMLVideoElement,
19690 };
19691 /**
19692  * pixi-adapter 统一入口
19693  * 根据运行环境（真机/模拟器）将 DOM 模拟对象挂载到全局
19694  *
19695  * 关键：真机环境中 IIFE bundle 的自由变量（document、window 等）
19696  * 必须在 JS 引擎的全局作用域中可达，仅挂载到 GameGlobal 不够--
19697  * GameGlobal 只是跨文件共享对象，不是全局作用域。
19698  * 因此真机需要挂载到 globalThis / global 上。
19699  */
19700 const platform = require('./platform');
19701 const { noop } = require('./util');
19702 const Image = require('./Image');
19703 const { canvas } = require('./canvas');
19704 const location = require('./location');
19705 const document = require('./document');
19706 const navigator = require('./navigator');
19707 const localStorage = require('./localStorage');
```

```
19708 const XMLHttpRequest = require('./XMLHttpRequest');
19709 const { registerTouchEvents } = require('./TouchEvent');
19710 const {
19711   Element,
19712   HTMLCanvasElement,
19713   HTMLImageElement,
19714   HTMLVideoElement,
19715 } = require('./element');
19716 // ===== 获取真正的 JS 全局对象 =====
19717 // 优先 globalThis (ES2020+) , 其次 global (Node/V8) , 最后 GameGlobal
19718 const _realGlobal = (typeof globalThis !== 'undefined' && globalThis)
19719   || (typeof global !== 'undefined' && global)
19720   || GameGlobal;
19721 // ===== Patch Object.defineProperty =====
19722 const _origDefineProperty = Object.defineProperty;
19723 Object.defineProperty = function safeDefineProperty(obj, prop, descriptor) {
19724   try {
19725     return _origDefineProperty.call(Object, obj, prop, descriptor);
19726   } catch (e) {
19727     if (e instanceof TypeError) return obj;
19728     throw e;
19729   }
19730 };
19731 const _origDefineProperties = Object.defineProperties;
19732 Object.defineProperties = function safeDefineProperties(obj, props) {
19733   for (const key in props) {
19734     if (Object.prototype.hasOwnProperty.call(props, key)) {
19735       try {
19736         _origDefineProperty.call(Object, obj, key, props[key]);
19737       } catch (e) {
19738         if (!(e instanceof TypeError)) throw e;
19739       }
19740     }
19741   }
19742   return obj;
19743 };
19744 // ===== 获取系统信息 =====
19745 const sysInfo = platform.getSystemInfoSync();
19746 const isDevtools = sysInfo.platform === 'devtools';
19747 // ===== 定时器 & 动画帧 polyfill =====
19748 // 真机 IIFE bundle 可能无法以自由变量访问这些 API,
19749 // 在 adapter 模块作用域中它们可用, 挂到真正的全局对象。
19750 ;(function _patchTimers() {
19751   var pairs = {};
19752   if (typeof setTimeout !== 'undefined') pairs.setTimeout = setTimeout;
19753   if (typeof clearTimeout !== 'undefined') pairs.clearTimeout = clearTimeout;
19754   if (typeof setInterval !== 'undefined') pairs.setInterval = setInterval;
19755   if (typeof clearInterval !== 'undefined') pairs.clearInterval = clearInterval;
19756   if (typeof requestAnimationFrame !== 'undefined') pairs.requestAnimationFrame = requestAnima...
19757   if (typeof cancelAnimationFrame !== 'undefined') pairs.cancelAnimationFrame = cancelAnimati...
```

```
19758 for (var k in pairs) {
19759     if (typeof _realGlobal[k] === 'undefined') _realGlobal[k] = pairs[k];
19760     if (typeof GameGlobal[k] === 'undefined') GameGlobal[k] = pairs[k];
19761 }
19762 }});
19763 // ===== 禁用 OffscreenCanvas =====
19764 if (typeof GameGlobal !== 'undefined') {
19765     GameGlobal.OffscreenCanvas = undefined;
19766     _realGlobal.OffscreenCanvas = undefined;
19767 }
19768 // ===== WebGL / Canvas2D 上下文构造函数 =====
19769 let _WebGLRenderingContext = {};
19770 try {
19771     const _tmpCanvas = platform.createCanvas();
19772     const _tmpGl = _tmpCanvas.getContext('webgl');
19773     if (_tmpGl) _WebGLRenderingContext = _tmpGl.constructor || {};
19774 } catch (e) { /* 忽略 */ }
19775 let _CanvasRenderingContext2D = {};
19776 try {
19777     const _tmpCanvas2 = platform.createCanvas();
19778     const _tmpCtx = _tmpCanvas2.getContext('2d');
19779     if (_tmpCtx) _CanvasRenderingContext2D = _tmpCtx.constructor || {};
19780 } catch (e) { /* 忽略 */ }
19781 // ===== DOMParser =====
19782 class DOMParser {
19783     parseFromString() {
19784         return { documentElement: new Element() };
19785     }
19786 }
19787 // ===== performance =====
19788 const _performance = typeof performance !== 'undefined' ? performance : {
19789     now: Date.now.bind(Date),
19790 };
19791 // ===== window 事件系统 =====
19792 const _windowListeners = {};
19793 var _winEvtLogCount = 0;
19794 function _windowAddEventListener(type, handler, options) {
19795     if (!_windowListeners[type]) _windowListeners[type] = [];
19796     _windowListeners[type].push(handler);
19797     _winEvtLogCount++;
19798     if (_winEvtLogCount <= 20) {
19799         console.log('[pixi-adapt] globalThis.addEventListener 注册:', type, '(共' + _windowListeners[t...
19800     }
19801 }
19802 function _windowRemoveEventListener(type, handler) {
19803     if (!_windowListeners[type]) return;
19804     const idx = _windowListeners[type].indexOf(handler);
19805     if (idx !== -1) _windowListeners[type].splice(idx, 1);
19806 }
19807 function _windowDispatchEvent(type, event) {
```

```
19808 const queue = _windowListeners[type];
19809 if (queue) {
19810     const copy = queue.slice();
19811     copy.forEach(handler => {
19812         try { handler(event); } catch (e) { console.error('[window event]', type, e); }
19813     });
19814 }
19815 }
19816 GameGlobal.__windowDispatchEvent = _windowDispatchEvent;
19817 // ===== 事件构造函数 =====
19818 function _PointerEvent(type, opts) { this.type = type; Object.assign(this, opts || {}); }
19819 function _TouchEventCtor(type, opts) { this.type = type; Object.assign(this, opts || {}); }
19820 function _MouseEvent(type, opts) { this.type = type; Object.assign(this, opts || {}); }
19821 // ===== URL / Blob =====
19822 const _URL = {
19823     createObjectURL: function() { return ""; },
19824     revokeObjectURL: function() {},
19825 };
19826 function _Blob() {}
19827 // ===== 所有需要挂载的全局属性 =====
19828 const _allGlobals = {
19829     window: null, // 下面特殊处理
19830     document: document,
19831     navigator: navigator,
19832     location: location,
19833     Image: Image,
19834     Element: Element,
19835     HTMLCanvasElement: HTMLCanvasElement,
19836     HTMLImageElement: HTMLImageElement,
19837     HTMLVideoElement: HTMLVideoElement,
19838     WebGLRenderingContext: _WebGLRenderingContext,
19839     CanvasRenderingContext2D: _CanvasRenderingContext2D,
19840     XMLHttpRequest: XMLHttpRequest,
19841     DOMParser: DOMParser,
19842     localStorage: localStorage,
19843     performance: _performance,
19844     canvas: canvas,
19845     ontouchstart: noop,
19846     addEventListener: _windowAddEventListener,
19847     removeEventListener: _windowRemoveEventListener,
19848     self: null, // 下面特殊处理
19849     PointerEvent: _PointerEvent,
19850     TouchEvent: _TouchEventCtor,
19851     MouseEvent: _MouseEvent,
19852     URL: _URL,
19853     Blob: _Blob,
19854 };
19855 if (isDevtools) {
19856     // ===== 模拟器环境 =====
19857     // window 已存在（浏览器环境），用 defineProperty 补充/覆盖
```



```
19858 const _win = typeof window !== 'undefined' ? window : GameGlobal;
19859 for (const key in _allGlobals) {
19860   if (key === 'window' || key === 'self') continue;
19861   try {
19862     const desc = Object.getOwnPropertyDescriptor(_win, key);
19863     if (!desc || !desc.configurable) {
19864       _origDefineProperty.call(Object, _win, key, { value: _allGlobals[key], configurable: true...
19865     }
19866   } catch (e) { /* 只读属性忽略 */ }
19867 }
19868 // 关键修复: 包装 window.addEventListener / removeEventListener
19869 // PixiJS EventSystem 在 globalThis(window) 上注册 pointermove / pointerup,
19870 // 但 adapter 通过 _windowListeners 分发触摸事件--两个系统完全隔离。
19871 // 包装后 handler 会同时进入 _windowListeners, adapter 的 dispatchToWindow 就能触达 PixiJS。
19872 try {
19873   var _nativeWinAdd = _win.addEventListener.bind(_win);
19874   var _nativeWinRemove = _win.removeEventListener.bind(_win);
19875   _win.addEventListener = function(type, handler, options) {
19876     _windowAddEventListener(type, handler, options);
19877     return _nativeWinAdd(type, handler, options);
19878   };
19879   _win.removeEventListener = function(type, handler, options) {
19880     _windowRemoveEventListener(type, handler);
19881     return _nativeWinRemove(type, handler, options);
19882   };
19883   console.log('[pixi-adapter] 模拟器 window.addEventListener 已包装');
19884 } catch (e) {
19885   console.warn('[pixi-adapter] 包装 window.addEventListener 失败:', e);
19886 }
19887 // document 属性补充
19888 try {
19889   for (const key in document) {
19890     const desc = Object.getOwnPropertyDescriptor(_win.document, key);
19891     if (!desc || !desc.configurable) {
19892       _origDefineProperty.call(Object, _win.document, key, { value: document[key], configurable...
19893     }
19894   }
19895 } catch (e) { /* 忽略 */ }
19896 } else {
19897   // ===== 真机环境 =====
19898   // 关键: 必须同时挂载到 _realGlobal (JS 引擎全局对象) 和 GameGlobal (跨文件共享)
19899   // 这样 IIFE bundle 中的自由变量 document、window 等才能正确解析
19900   // window = 全局对象自身 (模拟浏览器行为)
19901   _realGlobal.window = _realGlobal;
19902   GameGlobal.window = _realGlobal;
19903   // self = 全局对象自身
19904   _realGlobal.self = _realGlobal;
19905   GameGlobal.self = _realGlobal;
19906   // addEventListener/removeEventListener 必须强制覆盖:
19907   // 微信框架可能内置了无效版本, PixiJS EventSystem 在 self 上注册
```

```
19908 // pointermove/pointerup 依赖这些函数正确工作
19909 var _forceOverwrite = new Set(['addEventListener', 'removeEventListener']);
19910 for (const key in _allGlobals) {
19911   if (key === 'window' || key === 'self') continue;
19912   var val = _allGlobals[key];
19913   var force = _forceOverwrite.has(key);
19914   // 挂到真正的全局作用域
19915   if (force || typeof _realGlobal[key] === 'undefined') {
19916     try { _realGlobal[key] = val; } catch (e) { /* 忽略 */ }
19917   }
19918   // 同时挂到 GameGlobal
19919   if (force || typeof GameGlobal[key] === 'undefined') {
19920     try { GameGlobal[key] = val; } catch (e) { /* 忽略 */ }
19921   }
19922 }
19923 // 确认事件系统已正确挂载
19924 console.log('[pixi-adapter] 真机事件系统检查:',
19925   'globalThis.addEventListener === _windowAddEventListener', _realGlobal.addEventListener === ...
19926   ', self.addEventListener === _windowAddEventListener', (_realGlobal.self && _realGlobal.self...
19927 }
19928 // ===== 全局 canvas =====
19929 // 微信框架可能已将 canvas 设为只读属性, 需 try-catch 保护
19930 try { GameGlobal.canvas = canvas; } catch (e) { /* 已由框架设置 */ }
19931 try { _realGlobal.canvas = canvas; } catch (e) { /* 只读属性忽略 */ }
19932 // ===== navigator.userAgent =====
19933 try {
19934   if (_realGlobal.window && _realGlobal.window.navigator) {
19935     _realGlobal.window.navigator.userAgent = navigator.userAgent;
19936   }
19937 } catch (e) { /* 只读属性忽略 */ }
19938 // ===== 注册触摸事件 =====
19939 registerTouchEvents();
19940 console.log('[pixi-adapter] 初始化完成, 平台:', platform.name, ', 环境:', isDevtools ? '模拟器' : '真机');
19941 console.log('[pixi-adapter] _realGlobal === GameGlobal', _realGlobal === GameGlobal,
19942   ', typeof document:', typeof _realGlobal.document,
19943   ', typeof window:', typeof _realGlobal.window);
19944 /**
19945  * localStorage 模拟, 基于平台 Storage API
19946  */
19947 const platform = require('./platform');
19948 const localStorage = {
19949   getItem(key) {
19950     try {
19951       return platform.getStorageSync(key) || null;
19952     } catch (e) {
19953       return null;
19954     }
19955   },
19956   setItem(key, value) {
19957     try {
```

```
19958     platform.setStorageSync(key, value);
19959   } catch (e) {
19960     console.warn('[localStorage] setItem failed:', key, e);
19961   }
19962 },
19963 removeItem(key) {
19964   try {
19965     platform.removeStorageSync(key);
19966   } catch (e) {
19967     console.warn('[localStorage] removeItem failed:', key, e);
19968   }
19969 },
19970 clear() {
19971   console.warn('[localStorage] clear() not implemented in minigame');
19972 },
19973 get length() {
19974   return 0;
19975 },
19976 key() {
19977   return null;
19978 },
19979 };
19980 module.exports = localStorage;
19981 /**
19982  * location 对象模拟
19983  */
19984 const location = {
19985   href: 'game.js',
19986   protocol: 'https:',
19987   host: 'minigame',
19988   hostname: 'minigame',
19989   port: '',
19990   pathname: '/game.js',
19991   search: '',
19992   hash: '',
19993   origin: 'https://minigame',
19994 };
19995 module.exports = location;
19996 /**
19997  * navigator 对象模拟
19998  */
19999 const platform = require('./platform');
20000 let _sysInfo;
20001 try {
20002   _sysInfo = platform.getSystemInfoSync();
20003 } catch (e) {
20004   _sysInfo = {};
20005 }
20006 const navigator = {
20007   platform: _sysInfo.platform || 'unknown',
```

```
20008   language: _sysInfo.language || 'zh_CN',
20009   appVersion: '5.0 (MiniGame)',
20010   userAgent: 'Mozilla/5.0 (MiniGame; ' + (_sysInfo.platform || '') + ') PixiJS/7',
20011   onLine: true,
20012   maxTouchPoints: 10,
20013   vendor: "",
20014   product: "",
20015   productSub: "",
20016   hardwareConcurrency: 4,
20017   // PixiJS 可能会检查 gpu 信息
20018   gpu: "",
20019 };
20020 module.exports = navigator;
20021 /**
20022  * 平台抽象层 - 统一微信/抖音小游戏 API
20023  * 所有 adapter 模块通过此模块调用平台 API，不直接写 wx.xxx 或 tt.xxx
20024  */
20025 const _isWechat = typeof wx !== 'undefined';
20026 const _isDouyin = typeof tt !== 'undefined';
20027 const _api = _isWechat ? wx : _isDouyin ? tt : null;
20028 if (!_api) {
20029   console.error('[platform] 未检测到小游戏运行环境 (wx/tt) ');
20030 }
20031 const platform = {
20032   createCanvas: () => _api.createCanvas(),
20033   createImage: () => _api.createImage(),
20034   getSystemInfoSync: () => _api.getSystemInfoSync(),
20035   getStorageSync: (key) => _api.getStorageSync(key),
20036   setStorageSync: (key, data) => _api.setStorageSync(key, data),
20037   removeStorageSync: (key) => _api.removeStorageSync(key),
20038   request: (opts) => _api.request(opts),
20039   connectSocket: (opts) => _api.connectSocket(opts),
20040   onTouchStart: (cb) => _api.onTouchStart(cb),
20041   onTouchMove: (cb) => _api.onTouchMove(cb),
20042   onTouchEnd: (cb) => _api.onTouchEnd(cb),
20043   onTouchCancel: (cb) => _api.onTouchCancel(cb),
20044   offTouchStart: (cb) => _api.offTouchStart(cb),
20045   offTouchMove: (cb) => _api.offTouchMove(cb),
20046   offTouchEnd: (cb) => _api.offTouchEnd(cb),
20047   offTouchCancel: (cb) => _api.offTouchCancel(cb),
20048   createInnerAudioContext: () => _api.createInnerAudioContext(),
20049   name: _isWechat ? 'wechat' : _isDouyin ? 'douyin' : 'unknown',
20050   api: _api,
20051 };
20052 module.exports = platform;
20053 /**
20054  * 工具函数
20055  */
20056 function noop() {}
20057 module.exports = { noop };
```